SWIFT: Shallow and SIMD-Aware CKKS Functional Bootstrapping for Low-Latency

Jung Hee Cheon[1,2], Daehyun Jang[1], and Jaehee Kang[1]

[1] Department of Mathematical Sciences, Seoul National University, Seoul
{jhcheon, jadh0309, jaehee.kang422}@snu.ac.kr
[2] CryptoLab Inc. Seoul

Abstract. The CKKS homomorphic encryption scheme can achieve high throughput for large batches, typically of tens of thousands of inputs, whereas DM/CGGI schemes offer low latency for a single input. In existing CKKS approaches, the multiplicative depth required for nonlinear function evaluation demands a large ciphertext modulus, which forces a large ring degree regardless of the input size. Practical applications, such as encrypted MLP inference, may require only a few hundred function evaluations at a time. For these moderate batch sizes, existing CKKS-based methods requiring large multiplicative depth incur high latency without fully benefiting from their high throughput.

We present SWIFT, a CKKS functional-bootstrapping method that reduces latency by using multiple slots to lower multiplicative depth. For input x, SWIFT packs its integer multiples ℓx into unused slots. Exponential bootstrapping then generates the powers $\exp(2\pi \mathrm{i}\ell x) = \exp(2\pi \mathrm{i}x)^\ell = \alpha^\ell$ in parallel. Compared to previous approaches that compute these powers through homomorphic multiplications, SWIFT directly generates those powers and reduces the required multiplicative depth and ciphertext modulus, allowing a smaller ring degree and lower latency.

We also introduce a trigonometric approximation method for singular functions such as $1/\sqrt{x}$, whose derivative is unbounded near zero. We observe that the previous Fourier extension method of Bian et al. (EUROCRYPT'26) can produce extremely large coefficient norms for these functions when focusing solely on reducing the polynomial degree. Instead, we solve an optimization problem to find a trigonometric polynomial that minimizes the coefficient norm while satisfying the required approximation accuracy for the target function. The resulting trigonometric polynomial can be evaluated efficiently and with numerical stability using SWIFT.

Our implementation evaluates ReLU on $[-1, 1]$ and $1/\sqrt{x}$ on $[0.0005, 1]$ with 12-bit absolute and relative precision, respectively, at ring degree $\log N = 15$. For batches of 512 inputs, SWIFT achieves speedups of $3.94\times$ and $2.86\times$ over the corresponding CKKS baselines for ReLU and $1/\sqrt{x}$, respectively. Compared with latency estimates of DM/CGGI-based methods, SWIFT achieves lower latency for 8-to-8 LUT and 12-to-12 LUT at batch sizes as small as 32 inputs.

Keywords: Homomorphic Encryption, CKKS, Functional Bootstrapping.

1 Introduction

Homomorphic encryption (HE) enables computation over encrypted data [8, 11, 12, 15, 18, 19, 40]. Gentry constructed the first fully homomorphic encryption scheme by introducing bootstrapping, which refreshes ciphertexts to allow continued homomorphic computation [19]. Among various homomorphic encryption schemes, CKKS [11] supports approximate arithmetic on real or complex data using single instruction, multiple data (SIMD) packing, achieving high throughput typically with tens of thousands of inputs in parallel. CKKS is used in encrypted machine learning, secure data analysis, and other computations on encrypted numerical data. These applications often require the evaluation of nonlinear functions, such as activation functions and functions represented by lookup tables. Functional bootstrapping combines bootstrapping with the evaluation of a target function. CKKS functional bootstrapping [2,3,5,6,16] evaluates such lookup tables or activation functions by representing them as trigonometric polynomials in the exponential basis $e^{2\pi i \ell x}$ generated during bootstrapping, providing high throughput for these applications.

However, many applications require fewer inputs per batch than CKKS can accommodate, leaving much of its SIMD capacity unused while still incurring the latency of operating on large CKKS ciphertexts. In MLP-based tabular prediction or recommendation models, a layer may require only a few hundred activation-function evaluations in parallel. For example, a clinical tabular prediction model uses hidden-layer widths of 128, 64, and 32 [44], while DeepFM uses DNN layers with 200 or 400 neurons per layer [23]. DLRM similarly uses MLP hidden-layer widths such as 512, 256, and 64 [31]. Similar batch sizes arise in encrypted language models. For the 128×768 hidden-state matrices used by Rho et al. [39], each LayerNorm operation requires 128 inverse-square-root evaluations, one per token. These batch sizes are much smaller than the full SIMD capacity of CKKS, yet large enough for the lower throughput of DM/CGGI-based programmable bootstrapping. This leads us to the following question:

Can we make CKKS functional bootstrapping
latency-efficient for batch sizes of several hundred inputs?

The multiplicative depth required for polynomial evaluation is one structural cause of high latency in CKKS. In CKKS functional bootstrapping, the target function is approximated by a trigonometric polynomial. Lookup tables are typically represented using trigonometric Hermite interpolation [2,3,5], while smooth continuous functions can be approximated using Fourier extension techniques [6]. For a degree-d trigonometric polynomial approximating the target function, standard polynomial evaluation methods such as Paterson–Stockmeyer (PS) [38] require multiplicative depth $\log_2 d + O(1)$. Supporting greater multiplicative depth in CKKS requires a larger ciphertext modulus, which often necessitates a larger ring degree N to maintain a fixed security level. We refer to this as the *forced-large-ring problem*. The ring degree may be dictated by the degree of the polynomial being evaluated rather than by the target batch size.

A larger ring increases latency even when the application uses only a fraction of the available slots. We therefore seek a method that uses SIMD slots to reduce multiplicative depth and evaluation cost.

1.1 Our Contributions

Our contributions are twofold.

Low-Depth and Low-Latency CKKS Functional Bootstrapping. Consider evaluating a degree-d trigonometric polynomial at input x in CKKS. We first pack integer multiples ℓx into different slots. Then, applying 'exponential bootstrapping' generates the exponential powers $\exp(2\pi\mathrm{i} \cdot \ell x) = \alpha^\ell$ in parallel. We group these powers into t vectors. Then, contracting these t vectors with the coefficient tensor yields the trigonometric polynomial $\sum c_\ell \alpha^\ell$. This contraction needs multiplicative depth only $O(\log t)$, reducing the ciphertext modulus consumption and lowering ring degree at the same security level. This method also reduces the number of key-switching operations. Taking $t = 2$ as an example, the coefficient tensor is an $O(\sqrt{d}) \times O(\sqrt{d})$ matrix. The Halevi–Shoup baby-step–giant-step method [24] evaluates this PCMV using $O(d^{1/4})$ keyswitchings. The remaining computation is followed by $O(\log d)$ keyswitchings. Thus, polynomial evaluation uses a total of $O(d^{1/4})$ key switchings, which is smaller than $O(\sqrt{d})$ for the Paterson-Stockmeyer method.

Trigonometric Approximation with Numerical Stability. The Fourier extension method of Bian et al. [6] can produce large coefficient norms for singular functions such as $1/x$ and $1/\sqrt{x}$ on intervals close to their singularity at zero. Our numerical error analysis of SWIFT gives an error bound that grows linearly with the coefficient ℓ_1-norm. Motivated by this observation, we seek trigonometric approximations with small coefficient ℓ_1-norms. We propose directly finding a trigonometric polynomial that minimizes the coefficient norm subject to the prescribed approximation-error bound. We formulate this search as a convex optimization problem and solve it using existing optimization methods. As a result, we obtain approximations for singular functions such as $1/\sqrt{x}$ on $[0.0005, 1]$ with a degree-2^{11} trigonometric polynomial that enables evaluation with 12-bit relative precision. These approximations balance polynomial degree and coefficient norm, rather than focusing on degree as in the Fourier extension method. The cost of evaluating high-degree approximations is mitigated by the low-depth evaluation of SWIFT. Our implementation evaluates $1/\sqrt{x}$ in 2.41 and 3.19 seconds for batches of 256 and 512 inputs, respectively, achieving speedups of 3.78× and 2.86× over the prior CKKS baseline.

1.2 Technical Overview

Prior CKKS Functional Bootstrapping. We first review how existing CKKS functional bootstrapping works. Bootstrapping includes a step called EvalMod that recovers x from $x + qI$ by homomorphically evaluating reduction modulo q.

This step can be computed using $\frac{q}{2\pi}\sin\left(\frac{2\pi}{q}(x+qI)\right) = \frac{q}{2\pi}\sin\left(\frac{2\pi x}{q}\right) \approx x$ for x close to zero. By adjusting the scale, we can instead obtain sine (or cosine) of the normalized input x. Combining these values gives the periodic function $\exp(2\pi \mathrm{i} x) =: \alpha$. We can then evaluate a trigonometric polynomial by performing a further polynomial evaluation on α. We denote this 'exponential bootstrapping' procedure by $\mathsf{ExpBTS} : \mathsf{Enc}(x) \mapsto \mathsf{Enc}(\alpha)$. Existing methods use this approach to evaluate lookup tables through trigonometric Hermite interpolation [2, 3, 5], or to approximate continuous functions by trigonometric polynomials [6, 43]. However, after computing α, these approaches still evaluate a polynomial using the Paterson–Stockmeyer method, which consumes multiplicative depth logarithmic in the degree. This in turn requires a larger ciphertext modulus and can lead to a larger ring degree and higher latency.

Reducing Multiplicative Depth. As in the above approach, we seek an approximation $f(x) = T(\alpha)$ of the target function φ, where T is a degree-d Laurent polynomial in α. Using $\alpha^\ell = \exp(2\pi \mathrm{i} \ell x)$, we obtain

$$\varphi(x) \approx f(x) = \sum_{\ell=-d}^{d} c_\ell \alpha^\ell = \sum_{\ell=-d}^{d} c_\ell \exp(2\pi \mathrm{i} \ell x). \tag{1}$$

We generate the values $\exp(2\pi \mathrm{i} \ell x)$ in parallel by packing the scaled inputs ℓx into SIMD slots. A linear combination of these values with the plaintext coefficients c_ℓ gives $f(x)$. The depth therefore depends on evaluating the exponential map exp, rather than on generating its powers through successive multiplications.

Using Periodicity in Bootstrapping. The domain for evaluating $\exp(2\pi \mathrm{i} \ell x)$ depends on ℓ. If $x \in [-a, a]$, then $\ell x \in [-|\ell| a, |\ell| a]$. Thus, the interval on which we must approximate the exponential grows with $|\ell|$. Its required approximation degree may grow as well. We avoid this by using the periodicity of the exponential within CKKS bootstrapping. During CKKS bootstrapping, the ciphertext is converted into a coefficient representation at the bottom level. Ignoring numerical errors, each plaintext coefficient is represented modulo q/Δ, where q is the modulus and Δ is the scale. Since $\exp(2\pi \mathrm{i} x)$ has period one, we choose $q/\Delta = 1$. Thus, the scaled input ℓx is represented by a residue $r \equiv \ell x \pmod 1$ in the interval $[-1/2, 1/2)$. After modulus raising and conversion back to slots, the value has the form $r + k$ for an integer k, giving

$$\exp(2\pi \mathrm{i}(r+k)) = \exp(2\pi \mathrm{i} r) = \exp(2\pi \mathrm{i} \ell x) = \alpha^\ell.$$

For fixed bootstrapping parameters, the range of k is bounded independently of ℓ. Hence, the interval on which the exponential must be approximated no longer grows with $|\ell|$. Applied to an encryption of ℓx, it produces an approximation to α^ℓ. For real-valued f, we have $c_{-\ell} = \overline{c_\ell}$. Hence, it suffices to evaluate $z_{[d]}(\alpha) := \sum_{\ell=1}^{d} c_\ell \alpha^\ell$ and recover

$$f(x) = z_{[d]}(\alpha) + \overline{z_{[d]}(\alpha)} + c_0.$$

Various Batch Sizes via Tensor Decomposition. The construction above generates $v_{[d]} := (\alpha, \alpha^2, \ldots, \alpha^d)$ through a single call to ExpBTS. It then computes $z_{[d]}(\alpha) = \langle c_{[d]}, v_{[d]} \rangle$, where $c_{[d]} = (c_1, \ldots, c_d)$. Each input occupies d slots, so the maximum batch size is inversely proportional to d. To reduce the number of required slots per input, we use baby-step–giant-step (BSGS) decomposition. Choose sets B and G such that every index in $\{1, \ldots, d\}$ has a unique representation $b + g$, with $b \in B$ and $g \in G$. Define $v_B = (\alpha^b)_{b \in B}$ and $v_G = (\alpha^g)_{g \in G}$. Then,

$$z_{[d]}(\alpha) = \langle C v_G, v_B \rangle \tag{2}$$

for a matrix $C = (c_{b+g})_{b \in B, g \in G}$. Thus, ExpBTS now needs to generate only the powers in v_B and v_G. Choosing $|B| \approx |G| \approx \sqrt{d}$ reduces their number from d to approximately $2\sqrt{d}$. Evaluating the matrix-vector product Cv_G using the Halevi-Shoup baby-step-giant-step method [24], followed by the inner product with v_B, requires multiplicative depth two and $O(d^{1/4})$ key switchings.

We extend this decomposition to t factors. Choose $v_1, \ldots, v_t$ so that their tensor product contains the entries of $v_{[d]}$. With the corresponding coefficient tensor $\mathcal{C}$, we obtain

$$z_{[d]}(\alpha) = \langle \mathcal{C}, v_1 \otimes \cdots \otimes v_t \rangle.$$

For $t \geq 2$, this contraction requires depth $O(\log t)$. The choice of t therefore provides a tradeoff between batch size and evaluation cost. We also use conjugation to obtain $\overline{\alpha^\ell} = \alpha^{-\ell}$. This gives additional powers without separate bootstrapping evaluations. Section 3 describes how these powers are used to construct the tensor factors.

1.3 Related Work

CKKS Functional Bootstrapping over Discrete Domains. Alexandru et al. [2] and Bae et al. [5] extend CKKS bootstrapping to evaluate lookup tables (LUT) as trigonometric polynomials. In particular, their interpolation polynomials impose zero-derivative conditions at the evaluation points, reducing the effect of input perturbations. Alexandru et al. [3] instead impose conditions on the complex derivatives of the trigonometric polynomial and provide circuits that suppress a larger portion of the residual error.

Dumezy et al. [16] target LUTs with large input domains of size P. Their construction requires several sequential AKP-style functional-bootstrapping rounds for decomposing the input digit, and for large P, additional error cleaning. The subsequent evaluation step, however, requires only $O(\log \log P)$ multiplicative depth. This enables the evaluation of 18-bit LUTs with ring degree $\log_2 N = 16$. SWIFT uses the error reduction provided by Hermite interpolation, and does not add the error-cleaning steps used for larger LUTs. Both methods reduce multiplicative depth to permit a smaller ring degree N.

CKKS Functional Bootstrapping over Continuous Domains. Bian et al. [6] construct a periodic extension of the target function and approximate it using a

Fourier polynomial. For a sufficiently smooth extension, the approximation error decreases at a faster polynomial rate as the smoothness increases, enabling high-precision functional bootstrapping at lower degree. More recently, Xiao et al. [43] use trigonometric minimax approximation methods for CKKS functional bootstrapping.

DM/CGGI Programmable Bootstrapping. DM/CGGI-style programmable bootstrapping evaluates a lookup table while refreshing the input ciphertext [12,15]. Subsequent work improves the efficiency of higher-precision LUT evaluation through programmable and circuit bootstrapping [29,41,42]. For batched inputs, amortized bootstrapping methods pack multiple LWE ciphertexts into RLWE ciphertexts and bootstrap them together [14,21,22,27,30].

Shallow and Low-Latency CKKS Bootstrapping. Other work reduces the multiplicative depth or linear-transformation cost of CKKS bootstrapping itself [10, 13], allowing CKKS bootstrapping at ring degrees such as $\log_2 N = 14$ or $\log_2 N = 13$. These techniques reduce the cost of bootstrapping, while SWIFT reduces the cost of polynomial evaluation after the exponential map. Their linear transforms differ, but their periodic step can use the same exponential map. SWIFT can therefore use these bootstrapping methods to generate its t factors at a smaller ring degree. A recent paper by Ouyang et al. [33] builds upon SHIP and takes this approach.

Polynomial Evaluation Using SIMD Slots. Prior work uses plaintext slots to reduce polynomial-evaluation costs. Okada et al. [32] exploit polynomial factorization and Galois automorphisms in BFV/BGV. At a high level, if $P(x) = U(x)V(x)$, then evaluating $P(x)$ reduces to evaluating lower-degree polynomials $U(x)$ and $V(x)$ at different slots. The decomposed polynomial is accumulated by repeatedly applying a rotation and multiplying. Sylph [34] uses a similar but numerically stable method in CKKS. A recursive decomposition of the form $P = U^2 + V^2 - m$ reduces the evaluation of P to evaluating lower-degree polynomials U, V in parallel. With sufficient SIMD slots, these methods use $O(\log d)$ key switchings while retaining multiplicative depth $\Theta(\log d)$. SWIFT uses SIMD slots to reduce both operation counts and multiplicative depth.

Concurrent Work. In concurrent and independent work, Dumezy and Suvanto [17] propose an encoding that allows t-variate LUT evaluation at depth $O(\log t)$ (depth one for a univariate LUT), separately from bootstrapping. Their Appendix B converts ordinary discrete CKKS inputs into this encoding through bootstrapping. Like SWIFT, this conversion evaluates an exponential map during bootstrapping on scaled inputs to generate powers, and evaluation uses tensor products to reduce depth. Their method evaluates functions over finite domains; SWIFT combines the same power-generation technique with conjugation to evaluate univariate trigonometric polynomials over both discrete and continuous domains.

2 Preliminaries

2.1 Notation

For vectors, $\|$ denotes concatenation and $\odot$ denotes component-wise multiplication. We use $\otimes$ for the tensor product, represented as a one-dimensional vector. Equivalently, the Kronecker product. We write $u \cong v$, where $\cong$ denotes equality up to a permutation of coordinates. For real $x \in \mathbb{R}$, we write $\mathrm{i} := \sqrt{-1},\ \alpha = e^{2\pi \mathrm{i} x}$.

For a positive integer d, write $[d] := \{1, \ldots, d\}$. For $A \subset \mathbb{Z}$, let $-A := \{-a : a \in A\},\ \pm A := A \cup (-A)$. For $A_1, \ldots, A_t \subset \mathbb{Z}$, their (Minkowski) sum is $\sum_{i=1}^{t} A_i = A_1 + \cdots + A_t := \{a_1 + \cdots + a_t : a_i \in A_i\}$. We write $\bigoplus_{i=1}^{t} A_i = A_1 \oplus \cdots \oplus A_t = \Lambda$ when addition maps $A_1 \times \cdots \times A_t$ *bijectively* onto Λ.

2.2 Trigonometric Polynomial

Definition 1 (Real trigonometric polynomial space). *For an integer $d \geq 0$, the space of real trigonometric polynomials of degree at most d is defined by*

$$\mathcal{T}_d := \left\{ f(x) = c_0 + \sum_{\ell=1}^{d} \left(c_\ell e^{2\pi \mathrm{i} \ell x} + \overline{c_\ell} e^{-2\pi \mathrm{i} \ell x} \right) : c_0 \in \mathbb{R},\ c_\ell \in \mathbb{C} \right\}.$$

Due to the conjugate symmetry $c_\ell = \overline{c_{-\ell}}$, for any set $\Lambda \subset \pm[d]$ containing exactly one element from each pair $\{\ell, -\ell\}$, we have $f(x) = c_0 + z_\Lambda(\alpha) + \overline{z_\Lambda(\alpha)}$ where $z_\Lambda(\alpha) := \sum_{\ell \in \Lambda} c_\ell \alpha^\ell$. In our work, we write $c = (c_1, \ldots, c_d)$ and define the *coefficient (1-) norm* $\|c\|_1 := \sum_{\ell=1}^{d} |c_\ell|$, excluding c_0.

2.3 CKKS Scheme

CKKS [11] is an approximate homomorphic encryption scheme for complex vectors. For a power of two N, we write $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R}$. We call N the ring degree of $\mathcal{R}$. One CKKS ciphertext encrypts an $N/2$-dimensional complex vector. CKKS consists of the following PPT algorithms:

$$(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{add}, \mathsf{mult}, \mathsf{conj}, \mathsf{rot}).$$

- $\mathsf{KeyGen}(1^\lambda)$ outputs a secret key sk, a public key pk, and evaluation keys including a relinearization key rlk, rotation keys $\{\mathsf{rtk}_r\}$, and a conjugation key cjk.
- For a given message $\mathbf{m} \in \mathbb{C}^{N/2}$, Enc outputs a ciphertext $\mathsf{ct} = (a, b) \in \mathcal{R}_Q^2$ where $as + b \approx \mathrm{Ecd}(\Delta \mathbf{m}) \pmod{Q}$ for *scale factor* $\Delta \in \mathbb{Z}$, *modulus* $Q \in \mathbb{Z}$ where $\mathrm{Ecd} : \mathbb{C}^{N/2} \to \mathcal{R}$ is the encoding map. Dec recovers an approximation to $\mathbf{m}$ from ct.
- Homomorphic addition add and multiplication mult act slotwise; for these operations we use the usual notations $+$, $\cdot$. Multiplication between two ciphertexts uses keyswitchings using rlk.

– conj applies complex conjugation to every slot using cjk. We abbreviate $\mathsf{conj}(\mathsf{ct})$ as $\overline{\mathsf{ct}}$. rot_r cyclically moves the entry in slot j to slot $j+r \bmod (N/2)$ using rtk_r. These operations use keyswitching using $\mathsf{cjk}, \mathsf{rtk}_r$, respectively.

For $\mathsf{ct} \in \mathcal{R}_Q^2$, we call Q the ciphertext modulus. Operations on ciphertexts with a larger modulus cost more. Ciphertext multiplication and multiplication by a nonintegral plaintext consume modulus; exact multiplication by a ring integer, including a Gaussian integer, consumes no modulus. When the available moduli are discretized, we refer to each modulus as a level.

CKKS Bootstrapping. CKKS bootstrapping [9] refreshes a ciphertext so that further homomorphic operations can be performed. We write $\mathsf{StC}, \mathsf{CtS}$ for the homomorphic decoding and encoding maps, respectively, and $\mathsf{ModRaise}$ is a map that lifts a ciphertext to a larger modulus Q. CKKS bootstrapping starts as follows:

$$\begin{aligned}
& b_0 + a_0 s \approx \mathrm{Ecd}(\Delta \boldsymbol{m}) && (\bmod\ q_{\mathrm{in}}) \\
\xrightarrow{\mathsf{StC}}\ & b_1 + a_1 s \approx \Delta m(X) && (\bmod\ q_0) \\
\xrightarrow{\mathsf{ModRaise}}\ & b_1 + a_1 s \approx \Delta m(X) + q_0 I(X) && (\bmod\ Q) \\
\xrightarrow{\mathsf{CtS}}\ & b_2 + a_2 s \approx \mathrm{Ecd}(\Delta \boldsymbol{m} + q_0 \boldsymbol{I}) && (\bmod\ Q'),
\end{aligned}$$

where $\boldsymbol{m}$ and $\boldsymbol{I}$ are the coefficient vectors of $m(X)$ and $I(X)$, respectively. After CtS, a periodic function removes the dependence on $\boldsymbol{I}$. Conventional CKKS bootstrapping [9] uses an approximate modular reduction to recover the representative m:

$$b_2 + a_2 s \xrightarrow{\mathsf{EvalMod}} b_3 + a_3 s \approx \mathrm{Ecd}(\Delta \boldsymbol{m}) \pmod{Q''}.$$

When $|\Delta m| \ll q_0$, the original construction uses $\frac{q_0}{2\pi} \sin(2\pi(\Delta \boldsymbol{m} + q_0 \boldsymbol{I})/q_0) \approx \Delta \boldsymbol{m}$.

Other periodic functions can also be used. With $\Delta = q_0$, one can use the periodic function $\exp(2\pi \mathrm{i}(\boldsymbol{m} + \boldsymbol{I})) = \exp(2\pi \mathrm{i} \boldsymbol{m})$ to remove the effect of $\boldsymbol{I}$. In this case, the last stage can be evaluated as:

$$b_2 + a_2 s \xrightarrow{\mathsf{EvalExp}} b_3 + a_3 s \approx \mathrm{Ecd}\big(\Delta e^{2\pi \mathrm{i} \boldsymbol{m}}\big) \pmod{Q''}.$$

Current CKKS functional bootstrapping [2,5,6] uses this type of bootstrapping. In this work, we denote $\mathsf{ExpBTS} := \mathsf{EvalExp} \circ \mathsf{CtS} \circ \mathsf{ModRaise} \circ \mathsf{StC}$. The same description applies to SHIP-style bootstrapping [10,13], whose linear part differs but whose periodic step can be replaced by the same exponential map.

3 Polynomial Evaluation as a Tensor Contraction

Let $\varphi : I \to \mathbb{R}$ be the target function. We assume that $I \subseteq [-\frac{\sigma}{2}, \frac{\sigma}{2}]$ for some $0 < \sigma \leq 1$. If necessary, we apply an affine map to place the input domain within this interval. After this change of variables, we continue to denote the

input, domain, and target function by x, I, and φ, respectively. Let $f(x) \in \mathcal{T}_d$ be a degree-d real-valued trigonometric polynomial that approximates $\varphi(x)$ on I. For $\alpha = e^{2\pi i x}$, $f(x) \in \mathcal{T}_d$ can be written as $f(x) = \sum_{\ell=-d}^{d} c_\ell \alpha^\ell$, where $c_{-\ell} = \overline{c_\ell}$. If $\Lambda \subset \{-d, \ldots, -1, 1, \ldots, d\}$ contains exactly one element from each pair $\{\ell, -\ell\}$, then

$$z_\Lambda(\alpha) := \sum_{\ell \in \Lambda} c_\ell \alpha^\ell, \qquad f(x) = c_0 + z_\Lambda(\alpha) + \overline{z_\Lambda(\alpha)}. \tag{3}$$

Thus evaluating $f(x)$ reduces to evaluating $z_\Lambda(\alpha)$. Here and below, $\langle u, v \rangle = \sum_k u_k v_k$ denotes a bilinear contraction, without complex conjugation.

3.1 Tensor Contraction

Let us recall the *Baby-step-Giant-step* decomposition. Suppose that two sets $B, G \subset \mathbb{Z}$ satisfy $B \oplus G = \Lambda$. Let $v_B = (\alpha^b)_{b \in B}$, $v_G = (\alpha^g)_{g \in G}$. Then we have $v_B \otimes v_G \cong v_\Lambda$. For a $|B| \times |G|$ matrix $C = (c_{b+g})_{b \in B, g \in G}$, we have

$$z_\Lambda(\alpha) = \sum_{\ell \in \Lambda} c_\ell \alpha^\ell = \langle C v_G, v_B \rangle.$$

This decomposition extends to t factors, as in the following lemma.

Lemma 1 (Tensor decomposition). *Let $A_1, \ldots, A_t \subset \mathbb{Z}$ be finite. If $\bigoplus_{i=1}^{t} A_i = \Lambda$, then*

$$v_{A_1} \otimes \cdots \otimes v_{A_t} \cong v_\Lambda.$$

Consequently, for a t-tensor $\mathcal{C}_{a_1,\ldots,a_t} := c_{a_1+\cdots+a_t}$,

$$z_\Lambda(\alpha) = \langle \mathcal{C}, v_{A_1} \otimes \cdots \otimes v_{A_t} \rangle . \tag{4}$$

Proof. The coordinate of $v_{A_1} \otimes \cdots \otimes v_{A_t}$ indexed by $(a_1, \ldots, a_t)$ is $\alpha^{a_1} \cdots \alpha^{a_t} = \alpha^{a_1+\cdots+a_t}$. Since $A_1 \oplus \cdots \oplus A_t = \Lambda$, each exponent in Λ occurs exactly once. Reordering the coordinates therefore gives v_Λ. The coefficient in the corresponding coordinate of $\mathcal{C}$ is $c_{a_1+\cdots+a_t}$, so contraction gives $\sum_{\ell \in \Lambda} c_\ell \alpha^\ell = z_\Lambda(\alpha)$. □

3.2 Selecting Factor Sets

For each factor set $A_i \subset \mathbb{Z}_{>0}$, write $n_i := |A_i|$. Define $v_i := v_{A_i} = (\alpha^\ell)_{\ell \in A_i}$. Similarly, $\overline{v_i} = (\alpha^{-\ell})_{\ell \in A_i} = v_{-A_i}$, $v_i \| \overline{v_i} \cong v_{\pm A_i}$.

If the entries of v_i are generated by ExpBTS, CKKS conjugation gives $\overline{v_i}$. Using conjugation, we can therefore form the tensor $\bigotimes_{i=1}^{t} (v_i \| \overline{v_i})$, which has $\prod_i (2n_i)$ coordinates. The tensor cannot contain exactly the powers indexed by Λ: whenever $\ell \in \bigoplus_{i=1}^{t} (\pm A_i)$, $-\ell \in \bigoplus_{i=1}^{t} (\pm A_i)$ as well, whereas Λ contains exactly one member of every opposite pair. We choose factor sets A_i satisfying

$$\bigoplus_{i=1}^{t} (\pm A_i) = \pm[d] = \{-d, \ldots, -1, 1, \ldots, d\}, \tag{5}$$

where $d := \frac{1}{2}\prod_{i=1}^{t}(2n_i)$. As discussed in Section 2, the evaluation of a trigonometric polynomial requires only one member of each opposite pair $\{\ell, -\ell\}$, so we need only half of these exponents. Suppose that $A_1, \ldots, A_t$ satisfy 5. If $(a_1, \ldots, a_t)$ is the unique tuple such that $\sum_{i=1}^{t} a_i = \ell$, then $(-a_1, \ldots, -a_t)$ is the unique tuple that sums to $-\ell$. Exactly one of these tuples has its last coordinate in $+A_t$, whereas the other has its last coordinate in $-A_t$. Consequently, if we take

$$\Lambda := \left(\bigoplus_{i<t}(\pm A_i)\right) \oplus A_t \subset \pm[d],$$

then Λ contains one exponent from each opposite pair. Thus, $|\Lambda| = n_t \prod_{i<t}(2n_i) = d$. Lemma 1 gives

$$v_\Lambda \cong \left(\bigotimes_{i<t}(v_i \| \overline{v_i})\right) \otimes v_t.$$

It remains to choose $A_1, \ldots, A_t$ satisfying 5. The following proposition gives the construction, with $\Lambda = \{1, \ldots, d\}$. Appendix B gives the proof.

Proposition 1. *Let $t \geq 2$ and $n_1, \ldots, n_t \in \mathbb{Z}_{>0}$. Define $d := \frac{1}{2}\prod_{i=1}^{t}(2n_i)$.*

1. *There exist sets $A_i \subset \mathbb{Z}_{>0}$, with $|A_i| = n_i$, satisfying 5 if and only if at least one n_i is even.*
2. *Let n_t be even and define*

$$\begin{aligned} A_i &:= \frac{n_t}{2}\left(\prod_{j<i} 2n_j\right)\{1, 3, \ldots, 2n_i - 1\} \quad (1 \leq i < t), \\ A_t &:= \left\{\frac{n_t}{2}\left(\prod_{i<t} 2n_i - 1\right) + 1, \ldots, \frac{n_t}{2}\left(\prod_{i<t} 2n_i + 1\right)\right\}. \end{aligned} \tag{6}$$

For these sets, $\left(\bigoplus_{i<t}(\pm A_i)\right) \oplus A_t = \{1, \ldots, d\}$.

Remark 1. For $t = 1$, the construction is $A_1 = \{1, \ldots, d\}$ with $d = n_1$ and requires no parity condition.

3.3 Bilinear Form

The chosen sets $A_1, \ldots, A_t$ from (6) satisfy

$$z_{[d]}(\alpha) = \left\langle \mathcal{C}, \bigotimes_{i<t}(v_i \| \overline{v_i}) \otimes v_t \right\rangle, \qquad f(x) = c_0 + z_{[d]}(\alpha) + \overline{z_{[d]}(\alpha)}.$$

For $t \geq 2$, we group the factors into two vectors and write the contraction in bilinear form. Choose integers $b, g \geq 1$ with $b + g = t$, and define $B := \bigoplus_{i=1}^{b}(\pm A_i)$, $G := A_t \oplus \bigoplus_{b<j<t}(\pm A_j)$ with their corresponding tensor vectors

$$v_B := \bigotimes_{i=1}^{b}(v_i \| \overline{v_i}), \quad v_G := v_t \otimes \bigotimes_{b<j<t}(v_j \| \overline{v_j}). \tag{7}$$

Arranging the coefficients of $\mathcal{C}$ into an $|B| \times |G|$ matrix C gives

$$z_{[d]}(\alpha) = \langle Cv_G, v_B \rangle. \tag{8}$$

4 SWIFT: CKKS Evaluation Algorithm

In this section, we present SWIFT, a CKKS algorithm for evaluating a trigonometric polynomial $f \in \mathcal{T}_d$ using the tensor contraction in Section 3. We use sets $A_1, \ldots, A_t$ of equal size $|A_i| = n = 2^m \geq 2$ for $i \in [t]$. It follows that the degree d is $d = \frac{(2n)^t}{2} = 2^{t-1}n^t$. If the target polynomial has smaller degree, we can set the remaining coefficients to zero. The next figure shows the pipeline in this section.

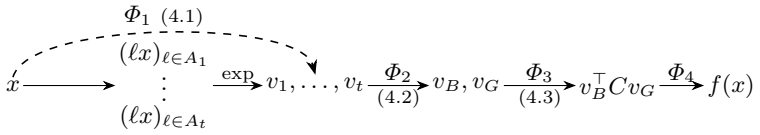


Fig. 1: Operations for evaluating $f(x)$ using the operations described in Section 4.

4.1 Factor Generation

Assume $n \mid N/2$, $2tn \leq N$, and set $\beta := \left\lfloor \frac{N}{2tn} \right\rfloor$, where β denotes the batch size. Given encrypted inputs x, we construct the t factor vectors $v_1, \ldots, v_t$. This procedure has three stages: (1) *packing*, (2) ExpBTS, and (3) *realignment*. Figure 2 illustrates these three stages with the slot layout. Algorithm 1 gives the corresponding ciphertext operations.

Let ct_x encrypt $x_0, \ldots, x_{\beta-1}$ in its first β slots and zero in all remaining slots. The *packing* stage places the values ℓx_j, for $\ell \in A_i$ and $i \in [t]$, in the CKKS slots specified below.

Next, we apply ExpBTS. If the input needs to be scaled to satisfy the assumption $I \subseteq [-\frac{\sigma}{2}, \frac{\sigma}{2}]$ in Section 3, we combine the affine map with the linear layers of StC. This requires no additional multiplicative depth.

Finally, the *realignment* step produces ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_t$ such that slot $j + kN/2n$ of ct_i encrypts $\alpha_j^{\ell_{i,k}}$, for $0 \leq j < \beta$ and $0 \leq k < n$. Thus, for every input x_j, ct_i stores the factor vector $(\alpha_j^\ell)_{\ell \in A_i}$.

4.2 Tensor Product

The tensor product in this subsection is used only when $t \geq 3$. Let $\mathsf{rot}_k(w)$ denote the cyclic rotation of an n-dimensional vector w by k coordinates. In the packed representation, this shift is implemented by the slot rotation $\mathsf{rot}_{kN/(2n)}$.

Algorithm 1 FactorGen

Require: A ciphertext ct_x whose leading β slots encrypt $x_0, \ldots, x_{\beta-1}$, and whose remaining slots encrypt zero.

Ensure: Ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_t$ such that, for $i \in [t]$, $0 \le j < \beta$, and $0 \le k < n$, slot $j + \frac{kN}{2n}$ of ct_i encrypts $\alpha_j^{\ell_{i,k}}$, where $\alpha_j = e^{2\pi i x_j}$.

Packing

$\mathsf{ct}_{x,i} \leftarrow \mathsf{rot}_{(i-1)\beta}(\mathsf{ct}_x) \quad (i = 1, \ldots, t)$

$\mathsf{ct}_{\mathrm{reg}} \leftarrow \sum_{r=1}^{t-1} \frac{n}{2}(2n)^{r-1}\mathsf{ct}_{x,r}$

$\mathsf{ct}_{\mathrm{base}} \leftarrow \mathsf{ct}_{\mathrm{reg}} + \left(\frac{d-n}{2} + 1\right)\mathsf{ct}_{x,t}$

$\mathsf{ct}_{\mathrm{step}} \leftarrow 2\mathsf{ct}_{\mathrm{reg}} + \mathsf{ct}_{x,t}$

for $p = 0, \ldots, m-1$ **do**

  $r_p \leftarrow \frac{2^p N}{2n}$, $\mathsf{ct}_{\mathrm{base}}^{(0)} \leftarrow \mathsf{ct}_{\mathrm{base}}$, $\mathsf{ct}_{\mathrm{step}}^{(0)} \leftarrow \mathsf{ct}_{\mathrm{step}}$

  $\mathsf{ct}_{\mathrm{base}} \leftarrow \mathsf{ct}_{\mathrm{base}}^{(0)} + \mathsf{rot}_{r_p}\left(\mathsf{ct}_{\mathrm{base}}^{(0)} + 2^p \mathsf{ct}_{\mathrm{step}}^{(0)}\right)$

  $\mathsf{ct}_{\mathrm{step}} \leftarrow \mathsf{ct}_{\mathrm{step}}^{(0)} + \mathsf{rot}_{r_p}\left(\mathsf{ct}_{\mathrm{step}}^{(0)}\right)$

end for

ExpBTS *and realignment*

$\mathsf{ct}_{\mathsf{pack}} \leftarrow \mathsf{ExpBTS}(\mathsf{ct}_{\mathrm{base}})$ ▷ Affine map may be absorbed into StC

$\mathsf{ct}_i \leftarrow \mathsf{rot}_{-(i-1)\beta}(\mathsf{ct}_{\mathsf{pack}}) \quad (i = 1, \ldots, t)$

return $(\mathsf{ct}_1, \ldots, \mathsf{ct}_t)$

Cost. For a batch of β inputs, all operations outside ExpBTS, except for the affine map, are $\mathbb{Z}$-linear and hence add no multiplicative depth. The algorithm uses one ExpBTS call, $2(t-1)+2\log_2 n$ rotations, and at most $t+\log_2 n-1$ integer scalar multiplications by values other than 0 and ± 1.

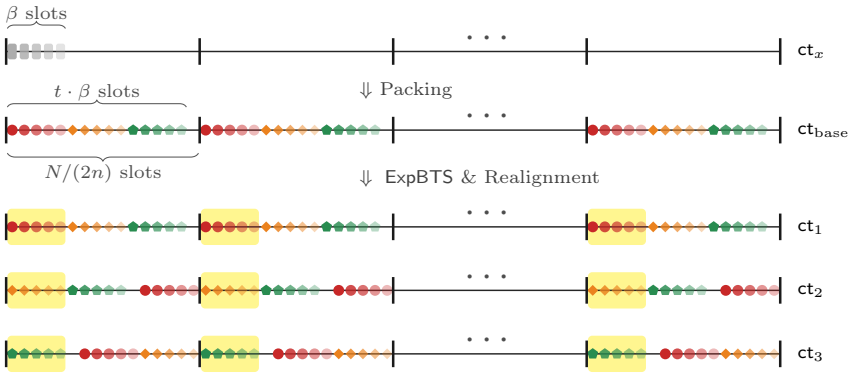


Fig. 2: Slot layout of the factor generation in Algorithm 1 for $t = 3$, $\beta = 5$, and $N/(2n) = 16$. The packed values are passed to ExpBTS before realignment.

The following algorithm evaluates a tensor product using rotations and ciphertext multiplications. Each ciphertext has ring degree N and stores the entries of an n-dimensional vector $N/(2n)$ slots apart.

Algorithm 2 Tensor: tensor product of ciphertext sequences

Require: Concatenated ciphertexts $\mathsf{CT}_w = \|_{i=1}^{a}\mathsf{ct}_{w,i}$ and $\mathsf{CT}_u = \|_{j=1}^{b}\mathsf{ct}_{u,j}$, where each ciphertext encrypts an n-dimensional vector. Each ciphertext has ring degree N and stores the vector entries $h = N/(2n)$ slots apart.

Ensure: *abn* ciphertexts CT_z representing $\mathsf{CT}_w \otimes \mathsf{CT}_u$, up to permutation.

1: Choose positive divisors n_1, n_2 of n with $n_1 n_2 = n$ that minimize $a(n_2-1)+b(n_1-1)$.
2: $\mathsf{ct}_{\rho^q_{w,i}} \leftarrow \mathsf{rot}_{-qn_1h}(\mathsf{ct}_{w,i})$ for $i \in [a]$ and $0 \le q < n_2$.
3: $\mathsf{ct}_{\rho^p_{u,j}} \leftarrow \mathsf{rot}_{ph}(\mathsf{ct}_{u,j})$ for $j \in [b]$ and $0 \le p < n_1$.
4: $\mathsf{CT}_z \leftarrow \|_{i\in[a],\ j\in[b],\ 0\le q<n_2,\ 0\le p<n_1}\big(\mathsf{ct}_{\rho^q_{w,i}} \cdot \mathsf{ct}_{\rho^p_{u,j}}\big)$.
5: **return** CT_z.

Cost. The algorithm outputs *abn* ciphertexts at depth one using $a(n_2-1)+b(n_1-1)$ rotations and *abn* ciphertext multiplications.

Algorithm 2 allows us to compute v_B and v_G in encrypted form. Let $b, g \in \mathbb{Z}_{\ge 1}$ satisfy $b+g=t$. For $i<t$, define

$$\mathsf{CT}_i := \mathsf{ct}_i \| \overline{\mathsf{ct}_i}, \qquad \mathsf{CT}_t := \mathsf{ct}_t.$$

Starting from the CT_i in each group, we apply Algorithm 2 along a balanced binary tree. This gives

$$\mathsf{CT}_B = \bigotimes_{1\le i\le b} \mathsf{CT}_i = \Big\|_{k=1}^{2(2n)^{b-1}} \mathsf{ct}_{B,k}, \qquad \mathsf{CT}_G = \bigotimes_{b<i\le t} \mathsf{CT}_i = \Big\|_{k=1}^{(2n)^{g-1}} \mathsf{ct}_{G,k}.$$

The two lists represent v_B and v_G, respectively, up to a fixed order. The constructions of CT_B and CT_G have multiplicative depths $\lceil \log_2 b \rceil$ and $\lceil \log_2 g \rceil$, respectively. The following proposition gives the asymptotic cost of this construction. Appendix C provides the proof.

Proposition 2 (Cost of tensor product). *Fix $b, g \ge 1$ with $b+g \ge 3$. The reconstruction of CT_B and CT_G uses*

$$\#\mathsf{ks} = 2(2n)^{b-1} + (2n)^{g-1} + o\Big((2n)^{\max\{b,g\}-1}\Big)$$

4.3 Evaluating the Bilinear Contraction

Write the reconstructed vectors as $v_B = (b_1\| \cdots \| b_R)$ and $v_G = (g_1\| \cdots \| g_S)$ for $R = 2(2n)^{b-1}$, $S = (2n)^{g-1}$, where every block has n coordinates. Write the coefficient matrix $C \in \mathbb{C}^{nR\times nS}$ as a block matrix $C = (C_{r,s})$, where each $C_{r,s} \in \mathbb{C}^{n\times n}$. Then with $u_r := \sum_{s=1}^{S} C_{r,s} g_s$, we have $z_{[d]}(\alpha) = \sum_{r=1}^{R} \langle u_r, b_r \rangle$.

We evaluate the plaintext-matrix/ciphertext-vector product Cv_G blockwise using the Halevi–Shoup technique [24]; denote this operation by PCMV. It returns concatenated ciphertexts $\mathsf{CT}_u = (\mathsf{ct}_{u,1}, \ldots, \mathsf{ct}_{u,R})$ representing Cv_G. For

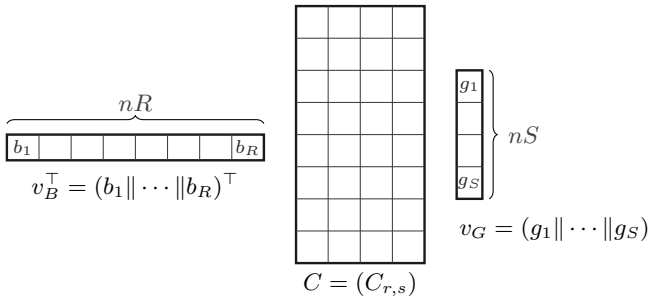


Fig. 3: Block structure of the bilinear term $\langle Cv_G, v_B\rangle = v_B^\top C v_G$. The plaintext matrix $C \in \mathbb{C}^{nR\times nS}$ is an $R \times S$ block matrix of $n \times n$ submatrices $C_{r,s} \in \mathbb{C}^{n\times n}$.

$n = n_{\mathrm{BS}} n_{\mathrm{GS}}$, it has multiplicative depth one and uses $S(n_{\mathrm{BS}}-1)+R(n_{\mathrm{GS}}-1)$ rotations and $nRS = d/n$ plaintext multiplications. Appendix C gives the diagonal method for each block and proves the cost bound.

For the encrypted inner product, first set $\mathsf{ct}_w := \sum_{r=1}^{R} \mathsf{ct}_{u,r} \cdot \mathsf{ct}_{B,r}$, and then sum its slots by repeated rotations and additions. This operation has multiplicative depth one and uses R ciphertext multiplications and $\log_2 n$ rotations. Let ct_z be the output of this procedure. Then $\mathsf{ct}_f := \mathsf{ct}_z + \overline{\mathsf{ct}_z} + c_0$ encrypts the result. The detailed procedure and the cost are given in Appendix C.

Proposition 3 (Cost of the bilinear contraction). *Choose $n_{\mathrm{BS}}, n_{\mathrm{GS}}$ with $n = n_{\mathrm{BS}} n_{\mathrm{GS}}$. Given the reconstructed block lists CT_B and CT_G, the computation of ct_f described above has keyswitching and plaintext multiplication costs*

$$\begin{aligned} \#\mathsf{ks} &= (2n)^{g-1}(n_{\mathrm{BS}}-1) + 2(2n)^{b-1} n_{\mathrm{GS}} + \log_2 n + 1, \\ \#\mathsf{ptmult} &= (2n)^{b+g-1} = (2n)^{t-1}. \end{aligned}$$

For fixed b, g and $n = 2^m$, the minimizing choices and the resulting key-switching counts are

$$\begin{aligned} b < g: &\quad n_{\mathrm{BS}} = 1, && \#\mathsf{ks} = (2n)^b && +o((2n)^b), \\ b = g: &\quad n_{\mathrm{BS}} = 2^{\lceil m/2\rceil}, && \#\mathsf{ks} = \left(2^{\lceil m/2\rceil} + 2^{\lfloor m/2\rfloor+1}\right)(2n)^{b-1} && +o((2n)^{b-1/2}), \\ b > g: &\quad n_{\mathrm{BS}} = n, && \#\mathsf{ks} = 2(2n)^{b-1} + \frac{1}{2}(2n)^g && +o((2n)^{b-1}). \end{aligned}$$

4.4 Total Cost of SWIFT

We summarize the cost of evaluating a trigonometric polynomial using SWIFT. We first choose the partition sizes b and g with $b + g = t$ to minimize the multiplicative depth. Computing $\langle Cv_G, v_B\rangle$ needs the multiplication depth $D = \max\{\lceil \log_2 b\rceil + 1, \lceil \log_2 g\rceil + 2\}$. If the available depth is D, we may take at most $b = 2^{D-1}$ and $g = 2^{D-2}$. Hence the maximum t at depth D is $t = b + g = \frac{3}{4}\cdot 2^D$,

and conversely, the minimum depth required by SWIFT with t tensor factors is $D_t := \left\lceil \log_2\left(\frac{4t}{3}\right) \right\rceil$.

Now choose b and g with $b+g=t$ to attain this minimized depth. Among such pairs, we choose one that minimizes the key-switching counts in Algorithm 1, Propositions 2 and 3. The following theorem gives these choices and the resulting evaluation costs. The proof is deferred to Appendix C.

Theorem 1 (Cost of SWIFT). *Fix $t \geq 2$, and let $n = 2^m \geq 2$, with $n \mid N/2$ and $2tn \leq N$, and let $d = (2n)^t/2$. Take $g = \min\left\{\left\lceil \frac{t}{2} \right\rceil, 2^{D_t-2}\right\}$ and set $b = t-g$. Then, the total key-switching count of SWIFT is asymptotically minimized, under the depth D_t. The resulting costs are*

$$\#\mathsf{ks} = \Theta\Big((2d)^{\frac{1}{t}\max\{b-1,(t-1)/2\}}\Big), \#\mathsf{ptmult} = (2d)^{1-1/t}, \#\mathsf{ExpBTS} = 1$$

For $t = 1$, we can just take $(b, g) = (0, 1)$, and the depth is one. The cost consists of a single ExpBTS call, $3\log_2 d$ key switchings, and one plaintext multiplication. The costs for $t = 1, 2, 3, 4$ are summarized in Table 1.

t	(b,g)	β	Depth	Key switchings	Pt. mults	ExpBTS calls
1	$(0,1)$	$N/(2d)$	1	$3\log_2 d$	1	1
2	$(1,1)$	$\lfloor N/(2(2d)^{1/2}) \rfloor$	2	$\Theta((2d)^{1/4})$	$(2d)^{1/2}$	1
3	$(2,1)$	$\lfloor N/(3(2d)^{1/3}) \rfloor$	2	$\Theta((2d)^{1/3})$	$(2d)^{2/3}$	1
4	$(2,2)$	$\lfloor N/(4(2d)^{1/4}) \rfloor$	3	$\Theta((2d)^{3/8})$	$(2d)^{3/4}$	1

Table 1: Batch size, multiplicative depth, and operation counts for $t \leq 4$.

4.5 Numerical Stability of SWIFT

We now study the numerical stability of SWIFT. As in other finite-precision computations, errors introduced during CKKS evaluation may be amplified by subsequent operations. These include errors from key switching, rescaling, and bootstrapping. Understanding how these errors propagate is important for correctness and parameter selection.

Let $f = \Phi_s \circ \cdots \circ \Phi_1$, where $\Phi_i : \mathbb{C}^{r_{i-1}} \to \mathbb{C}^{r_i}$ is the map evaluated at stage i. For each i, let $\widetilde{\Phi}_i$ denote the finite-precision sub-circuit used to compute Φ_i. Composing these sub-circuits gives:

$$\widetilde{f} := \widetilde{\Phi}_s \circ \cdots \circ \widetilde{\Phi}_1,$$

the finite-precision circuit used to compute f. Then, the *numerical error* at input x is defined as

$$\eta_{\text{num}}(x) := |f(x) - \widetilde{f}(x)|.$$

Error Analysis. Set $\widetilde{x}_0 := x \in \mathbb{C}^{r_0}$ as the input. For $1 \le i \le s$, define

$$\widetilde{x}_i := (\widetilde{\Phi}_i \circ \cdots \circ \widetilde{\Phi}_1)(x), \quad e_i := \Phi_i(\widetilde{x}_{i-1}) - \widetilde{\Phi}_i(\widetilde{x}_{i-1}), \quad g_i := \Phi_s \circ \cdots \circ \Phi_{i+1}$$

where g_s is the identity function. Then $f(x) - \widetilde{f}(x)$ is

$$\begin{aligned} f(x) - \widetilde{f}(x) = g_1(\Phi_1(\widetilde{x}_0)) - g_s(\widetilde{\Phi}_s(\widetilde{x}_{s-1})) &= \sum_{i=1}^{s} \left[g_i\big(\Phi_i(\widetilde{x}_{i-1})\big) - g_i\big(\widetilde{\Phi}_i(\widetilde{x}_{i-1})\big) \right] \\ &= \sum_{i=1}^{s} \left[g_i\big(\widetilde{x}_i + e_i\big) - g_i\big(\widetilde{x}_i\big) \right]. \end{aligned}$$

This formula reduces the error analysis to bounding the s terms above. Now, we use Lipschitz bounds to obtain a worst-case bound for each term.[3] For a map $g : X \to Y$ between normed spaces X and Y, a *Lipschitz bound of g over* $Z \subseteq X$ is the smallest number $L \ge 0$ such that $\|g(u) - g(v)\| \le L\|u - v\|$ for all $u, v \in Z$. We write $\mathrm{Lip}_Z(g)$ for such L. Thus, applying the triangle inequality and the Lipschitz bound gives

$$\eta_{\mathrm{num}}(x) = |f(x) - \widetilde{f}(x)| \le \sum_{i=1}^{s} \mathrm{Lip}_{Z_i}(g_i)\|e_i\|, \tag{9}$$

where each Lipschitz bound is taken over a set Z_i containing $\widetilde{x}_i$ and $\widetilde{x}_i + e_i$. Thus, bounding the numerical error reduces to bounding e_i and $\mathrm{Lip}_{Z_i}(g_i)$ for $1 \le i \le s$. We use the ℓ_∞ norm for vectors and Lipschitz bounds.

Application to SWIFT. As shown in Figure 1, SWIFT evaluates f at a real input x as a composition of four maps, $f = \Phi_4 \circ \Phi_3 \circ \Phi_2 \circ \Phi_1$. For $t \ge 2$, these maps are:

$$\underbrace{x}_{\in\mathbb{R}} \overset{\Phi_1}{\longmapsto} \underbrace{(v_1, \ldots, v_t)}_{\in\mathbb{C}^{nt}} \overset{\Phi_2}{\longmapsto} \underbrace{(v_B, v_G)}_{\in\mathbb{C}^{n(R+S)}} \overset{\Phi_3}{\longmapsto} \underbrace{v_B^\top C v_G}_{\in\mathbb{C}} \overset{\Phi_4}{\longmapsto} \underbrace{f(x)}_{\in\mathbb{C}}.$$

For $t = 1$, we use the same four maps by taking Φ_2 to be the identity map, with $\Phi_2(v_1) = v_G = v_1$ and $\Phi_3(v_G) := c^\top v_G$, without v_B, where $c = (c_1, \ldots, c_d)$.

The map $g_3 = \Phi_4$, given by $z \mapsto z + \bar{z} + c_0$, has Lipschitz bound at most 2, and g_4 is the identity with Lipschitz bound 1. Then, the inequality (9) gives

$$\eta_{\mathrm{num}}(x) \le \mathrm{Lip}_{Z_1}(g_1)\|e_1\|_\infty + \mathrm{Lip}_{Z_2}(g_2)\|e_2\|_\infty + 2|e_3| + |e_4|.$$

To bound the expression above, we make the following assumptions. Let $\epsilon_{\mathsf{BTS}} > 0$ bound the slotwise error introduced by ExpBTS. For each arithmetic operation other than ExpBTS, let $\epsilon_m > 0$ bound the total slotwise error introduced at its output, including mult, rot, conj. In all the following asymptotic

[3] A probabilistic bound may be used instead of a worst-case bound.

notation, we treat d, $\|c\|_1 = \sum_{\ell=1}^{d} |c_\ell|$, ϵ_{BTS}, and ϵ_m as variables, and t as a fixed constant. Throughout the following analysis, we assume $\|c\|_1 = \Omega(1)$ and $\epsilon_{\mathsf{BTS}} + d^{1+1/t}\epsilon_m = o(1)$.

Under the above assumptions, we first bound each $e_i \in \mathbb{C}^{r_i}$. The following proposition bounds the e_i, with proof given in Appendix D.

Proposition 4 (Local error bounds). *Suppose that each* ExpBTS *output error is bounded by ϵ_{BTS} and that every other operation satisfies the ϵ_m-bound above. For $t \geq 2$,*

$$\|e_1\|_\infty = O\left(\epsilon_{\mathsf{BTS}} + d^{1+1/t}\epsilon_m\right), \quad \|e_2\|_\infty = O\left(\epsilon_m\right),$$
$$|e_3| = O\Big((d^{1/t}\|c\|_1 + d)\epsilon_m\Big), \qquad |e_4| = O(\epsilon_m).$$

If $t = 1$, then $e_2 = 0$ and $|e_3| = O(d\epsilon_m)$, while the bounds for $\|e_1\|_\infty$ and $|e_4|$ remain unchanged.

Next, for $i = 1, 2$, we bound $\mathrm{Lip}_{Z_i}(g_i)$ with $Z_i \ni \widetilde{x}_i, \widetilde{x}_i + e_i$. We first choose the domains Z_i for the Lipschitz bounds by bounding the magnitudes of $\widetilde{x}_i$ and $\widetilde{x}_i + e_i$. Fix a real input x and let $t \geq 2$. Recall that

$$\widetilde{x}_1 = \widetilde{\Phi}_1(x), \quad \widetilde{x}_1 + e_1 = \Phi_1(x) = (v_1, \ldots, v_t),$$
$$\widetilde{x}_2 = \widetilde{\Phi}_2(\widetilde{x}_1), \quad \widetilde{x}_2 + e_2 = \Phi_2(\widetilde{x}_1),$$

where Φ_1 generates the factors and Φ_2 forms the tensor products. Define

$$\delta := \max\{1, \|\widetilde{x}_1\|_\infty, \|\widetilde{x}_1 + e_1\|_\infty, \|\widetilde{x}_2\|_\infty, \|\widetilde{x}_2 + e_2\|_\infty\} - 1,$$

and let $\mathbb{D}_\delta := \{z \in \mathbb{C} : |z| \leq 1 + \delta\}$. Then

$$\widetilde{x}_1, \widetilde{x}_1 + e_1 \in \mathbb{D}_\delta^{nt}, \qquad \widetilde{x}_2, \widetilde{x}_2 + e_2 \in \mathbb{D}_\delta^{n(R+S)}.$$

The following lemma and proposition give the Lipschitz bounds, with proofs given in Appendix D.

Lemma 2 (Bound for δ). *Let $t \geq 2$ be fixed. Then,*

$$\delta \leq (1 + \|e_1\|_\infty)^t (1 + \epsilon_m)^{4t} - 1 = O\Big(\epsilon_{\mathsf{BTS}} + d^{1+1/t}\epsilon_m\Big) = o(1).$$

Proposition 5 (Lipschitz bounds). *Let $t \geq 2$ be fixed. Then*

$$\mathrm{Lip}_{\mathbb{D}_\delta^{nt}}(g_1) \leq 2t(1+\delta)^{t-1}\|c\|_1 = (2t + o(1))\|c\|_1 = O(\|c\|_1),$$
$$\mathrm{Lip}_{\mathbb{D}_\delta^{n(R+S)}}(g_2) \leq 4(1+\delta)\|c\|_1 = (4 + o(1))\|c\|_1 = O(\|c\|_1).$$

For $t = 1$, both Lipschitz constants are at most $2\|c\|_1 = O(\|c\|_1)$.

Applying the above two propositions to the inequality (9) gives the following.

Theorem 2 (Numerical Error of SWIFT). *Let $t \geq 1$ be fixed. For every real input x,*

$$\eta_{\mathrm{num}}(x) \leq O(\|c\|_1)\,\epsilon_{\mathsf{BTS}} + O\Big(d^{1+1/t}\|c\|_1\Big)\,\epsilon_m.$$

Both terms are proportional to the coefficient norm $\|c\|_1$. Thus, for a fixed degree, $\|c\|_1$ is a key factor in numerical stability.

5 Trigonometric Polynomial Construction

In this section, we construct trigonometric polynomials that approximate a target function $\varphi : I \to \mathbb{R}$. We distinguish between continuous and discrete domains and choose an appropriate polynomial for each case.

For a continuous domain I, let $I = [-\frac{\sigma}{2}, \frac{\sigma}{2}]$ with $0 < \sigma < 1$, and assume that φ is continuous on I. The condition $\sigma < 1$ leaves room within a period of length 1 for endpoint matching. We seek a real trigonometric polynomial f satisfying

$$|f(x) - \varphi(x)| \le \eta_{\mathrm{app}} \qquad \text{for every } x \in I,$$

where $\eta_{\mathrm{app}} > 0$ is a prescribed approximation-error bound. For relative error, we instead require $|f(x) - \varphi(x)| \le \eta_{\mathrm{app}}|\varphi(x)|$ for every $x \in I$.

For a discrete domain, consider $I = \frac{1}{p}\mathbb{Z}/\mathbb{Z}$, where $p \ge 2$ is an integer. Let $0 < e_{\mathrm{pert}} < 1/(2p)$ be the allowed input-perturbation bound. We seek a real trigonometric polynomial f satisfying

$$|f(\widetilde{x}) - \varphi(x)| \le \eta_{\mathrm{in}} \qquad \text{for every } x \in I, |\widetilde{x} - x| < e_{\mathrm{pert}},$$

where $\eta_{\mathrm{in}} > 0$ is a prescribed input-error bound. Relative error can also be considered for lookup tables, but we do not consider it here, as their output values typically span a limited range. Among all such polynomials f, we prefer a low degree d to reduce the computational cost of SWIFT and also a low coefficient norm $\|c\|_1$ to limit the amplification of numerical errors.

5.1 Continuous Domain

We first review the smooth Fourier extension [6] as one method for constructing f and show that it can lead to substantial coefficient growth. We then formulate an optimization problem that effectively reduces the coefficient norm while meeting the required approximation-error bound η_{app}.

Smooth Fourier Extension. Let $\bar{\kappa} \ge 0$ be an integer and let $\varphi \in C^{\bar{\kappa}}(I)$. For each integer $0 \le \kappa \le \bar{\kappa}$, the Fourier extension method [6] constructs a 1-periodic function $\widehat{\varphi}_\kappa \in C^\kappa(\mathbb{R})$ that agrees with φ on I, using a C^κ bridge to match φ and its derivatives through order κ at the two endpoints of I. It then approximates φ on I by the degree-d Fourier truncation $f_{\kappa,d} \in \mathcal{T}_d$ of $\widehat{\varphi}_\kappa$, given by

$$c_{\kappa,\ell} := \int_0^1 \widehat{\varphi}_\kappa(u) e^{-2\pi \mathrm{i}\ell u}\, du, \qquad f_{\kappa,d}(x) := \sum_{|\ell| \le d} c_{\kappa,\ell} e^{2\pi \mathrm{i}\ell x}. \tag{10}$$

Write $\|c_\kappa\|_1 := \sum_{\ell=1}^{d} |c_{\kappa,\ell}|$.

For these extensions, the approximation error decays as $O(d^{-\kappa-2})$ away from singularities. Increasing κ improves this asymptotic rate, but may also enlarge the associated constant. Thus, a larger κ does not necessarily reduce the degree needed to meet a prescribed approximation-error bound. The Fourier extension

method compares the required degrees for different extension orders and selects the order yielding the smallest degree.

However, minimizing the degree alone can result in a large coefficient norm. In particular, differentiating (10) κ times gives

$$\left\| f_{\kappa,d}^{(\kappa)} \right\|_{L^\infty(\mathbb{R})} \leq 2(2\pi d)^\kappa \|c_\kappa\|_1.$$

Consequently, a large κ-th derivative of $f_{\kappa,d}$ imposes a large lower bound on $\|c_\kappa\|_1$. Increasing d relaxes this lower bound, although it does not by itself guarantee a smaller coefficient norm.

Because the extension is constructed by matching endpoint derivatives, targets with rapidly growing derivatives near an endpoint of I can be particularly prone to this tradeoff. Examples include $x \mapsto 1/x$, $x \mapsto 1/\sqrt{x}$, and $x \mapsto \sqrt{x}$ on intervals close to zero. For such targets, a nearby singularity or unbounded derivative causes the endpoint derivatives to grow rapidly with κ. Enforcing these increasingly large derivatives in the higher-order extension can in turn lead to Fourier truncations $f_{\kappa,d}$ with large κ-th derivatives and coefficient norms.

Table 2 illustrates this behavior for $x \mapsto 1/x$. Although the smallest degree is attained at $\kappa = 6$, the corresponding coefficient norm is substantially larger than those obtained with lower extension orders. Hence, selecting the minimum-degree extension order can lead to considerable numerical-error amplification.

κ	d_κ	$\|c\|_1$	κ	d_κ	$\|c\|_1$
1	258	46.45	6	**57**	3.768×10^5
2	109	257.2	7	58	2.368×10^6
3	74	1.552×10^3	8	57	1.489×10^7
4	63	9.607×10^3	9	60	9.358×10^7
5	58	6.005×10^4	10	61	5.882×10^8

Table 2: Minimum Fourier degrees and corresponding coefficient norms for κ-smooth periodic extensions of $x \mapsto 1/x$ on $[0.04, 1]$, affinely normalized to $I = [-1/4, 1/4]$ with $\sigma = 1/2$, subject to the relative approximation-error bound $\eta_{\text{app}} = 2^{-12}$.

Convex Optimization on the Target Interval As shown before, enforcing the smoothness at the endpoint can produce large coefficient norms when the degree is not sufficiently high. We instead optimize directly over $\mathcal{T}_d$, imposing the approximation requirement only on the target interval.

SWIFT's tensor factorization method is most efficient when the degree is a power of two. Thus, we use the candidate degree set $\mathcal{D} := \{2^j : j \in \mathbb{Z}_{\geq 0}\}$. Let's fix $d \in \mathcal{D}$ and an interval $I_\sigma = I = [-\frac{\sigma}{2}, \frac{\sigma}{2}]$ of length $\sigma \in (0, 1)$, with $\varphi_\sigma : I_\sigma \to \mathbb{R}$

a target function. Now consider the following optimization problem:[4]

$$C_d(\sigma) := \min_{f \in \mathcal{T}_d} \|c\|_1 \quad \text{subject to} \quad \|f(x) - \varphi_\sigma(x)\|_{\infty, I_\sigma} \le \eta_{\text{app}}.$$

If no feasible f exists, we set $C_d(\sigma) := \infty$. For fixed d, σ, η_{app}, and φ, this optimization problem is convex: the objective $\|c\|_1$ is convex in the coefficients, and the approximation-error constraint defines a convex feasible set because f depends linearly on the coefficients. We solve this convex optimization problem using a sequence of finite second-order cone programs (SOCPs) [28]. The specific procedure is described in Appendix E.

We select the approximating polynomial for SWIFT as follows:

1. Choose a finite set $\Sigma \subset (0, 1)$ of candidate interval lengths σ.
2. For each $d \in \mathcal{D}$, compute $C_d := \min_{\sigma \in \Sigma} C_d(\sigma)$. If $C_d < \infty$, let σ_d attain this minimum and let f_d be an optimizer of $C_d(\sigma_d)$. Otherwise, discard d as infeasible over Σ.
3. Compare the feasible pairs (d, C_d) and select the final approximation based on evaluation cost and numerical-error amplification.

We apply this procedure to the reciprocal function $x \mapsto 1/x$ considered in Table 2. The results are reported in Table 3. Under the same relative approximation-error bound, the Fourier extension with $\kappa = 6$ in Table 2 has degree 57 and coefficient norm 3.768×10^5, whereas the directly optimized candidate has degree 32 and coefficient norm 20.912. In this example, directly minimizing the coefficient norm over the target interval, without imposing endpoint-matching conditions, substantially reduces both the degree and the coefficient norm.

d	8	16	32	64	128
σ_d		0.70	0.85	0.85	0.80
C_d	∞	876.49	20.912	12.837	11.192

Table 3: Results of the target-interval convex optimization for $x \mapsto 1/x$ on $[0.04, 1]$. For each power-of-two degree d, the table reports C_d and σ_d, where $\Sigma = \{0.50, 0.55, \ldots, 0.90\}$ and the relative approximation-error bound is $\eta_{\text{app}} = 2^{-12}$.

5.2 Discrete Domain

Several works on interpolation methods have studied bounds on η_{in}. We use the first-order trigonometric Hermite interpolation polynomial of Alexandru et

[4] For relative error, we instead impose $\|(f(x) - \varphi_\sigma(x))/\varphi_\sigma(x)\|_{\infty, I_\sigma} \le \eta_{\text{app}}$, assuming $\inf_{x \in I_\sigma} |\varphi_\sigma(x)| > 0$.

al. [2], whose first derivative vanishes at the interpolation points. The following proposition bounds its degree, coefficient norm, and input error. The proof is deferred to Appendix D.4.

Proposition 6. *Let $\varphi : \frac{1}{p}\mathbb{Z}/\mathbb{Z} \to [0,1]$, and let f be its first-order trigonometric Hermite interpolation [2]. Then, for every $x \in I$ and every $\tilde{x}$ satisfying $|\tilde{x} - x| \le e_{\mathrm{pert}}$, we have $d \le p-1$, $\|c\|_1 \le \frac{\sqrt{p-1}}{4}$, and*

$$|\varphi(x) - f(\tilde{x})| \le \frac{e_{\mathrm{pert}}^2}{2} \sup_{\substack{x \in I \\ |u-x| \le e_{\mathrm{pert}}}} |f''(u)| = O(p^3 e_{\mathrm{pert}}^2).$$

One can also use higher-order Hermite interpolation instead. Sparse Hermite interpolation [3] reduces the effect of input perturbations and suppresses bootstrapping error by cancelling low-order terms in $T(\alpha + e) - T(\alpha)$. Since SWIFT generates several α^j separately, this error-cleaning argument does not directly apply to the bootstrapping error in SWIFT, but the effect of the initial perturbations in x still vanishes to higher order. A further analysis of this setting is of interest, as in SWIFT we only need to generate $\{\alpha^j\}_{j \in \Lambda}$ for 'sparse' Λ. We leave the study of other interpolation methods for SWIFT as future work.

6 Implementation

In this section, we report our implementation results. We compare the performance of SWIFT for different numbers of tensor factors t and identify the batch sizes for which SWIFT achieves the lowest total latency.

6.1 Experimental Tasks

Our experiments treat eight tasks: 4-to-4 LUT, 8-to-8 LUT, 12-to-12 LUT, sigmoid, tanh, ReLU, $1/x$, and $1/\sqrt{x}$. For each k-bit LUT, we use one instance whose outputs are sampled independently and uniformly from $2^{-k}\mathbb{Z}/\mathbb{Z}$.

For each task, we specify the input domain J, target error $\eta(x)$, and input perturbation bound e_{pert}, and baseline methods used for comparison. For the LUT tasks, $J = I = 2^{-k}\mathbb{Z}/\mathbb{Z}$. For the continuous-domain tasks, we affinely map J onto $I = [-\sigma/2, \sigma/2]$ and set $e_{\mathrm{pert}} = 0$. Our goal is to satisfy

$$|\varphi(x) - \widetilde{f}(\widetilde{x})| < \eta(x)$$

for every $x \in I$ and every $\widetilde{x}$ satisfying $|\widetilde{x} - x| \le e_{\mathrm{pert}}$ in the discrete-domain case, or

$$|\varphi(x) - \widetilde{f}(x)| < \eta(x)$$

for every $x \in I$ in the continuous-domain case. In both cases, we set $\eta(x) = \eta$ for absolute error and $\eta(x) = \eta|\varphi(x)|$ for relative error, where $\eta > 0$ is a constant. For continuous-domain tasks, we choose $\eta_{\mathrm{app}}(x) < \eta(x)$ and use $\eta_{\mathrm{app}}(x)$ as the approximation-error bound in the convex optimization problem in Section 5.

Discrete Domain. For 4-to-4 LUT and 8-to-8 LUT, our CKKS baseline is Alexandru et al.'s method [2]. For each k-to-k LUT, the input domain is $2^{-k}\mathbb{Z}/\mathbb{Z}$, with input perturbation at most 2^{-2k}. We choose $\eta(x) = 2^{-k-1}$ to ensure correct rounding. Alexandru et al. [2] evaluate a trigonometric Hermite polynomial using the Paterson–Stockmeyer algorithm; SWIFT uses the same interpolation method. We also include DM/CGGI programmable bootstrapping as a baseline: TFHE-rs for 4-to-4 LUT and Tetris [41] for 8-to-8 LUT, a DM/CGGI-based programmable bootstrapping method which supports LUTs with larger input domains. We also include BatchBoot [27], which reduces the amortized cost of TFHE-based LUT evaluation for large batches. We omit sparse Hermite interpolation [3] from the baselines because it is designed to provide sufficient error cleaning at the cost of additional multiplicative depth, making a direct comparison difficult.

For 12-to-12 LUT, we compare with Dumezy et al. [16]. The input domain is $2^{-12}\mathbb{Z}/\mathbb{Z}$, with input perturbation at most 2^{-24}. We choose $\eta(x) = 2^{-13}$ to ensure correct rounding. Dumezy et al.'s CMT-FBT method [16], with digit base 2^3, decomposes the 12-bit input into four 3-bit digits, and combines them through Kronecker (tensor) products. We also include Tetris [41] as a baseline.

Continuous Domain. For each continuous task below, we denote the original input domain by J. For SWIFT, we affinely map J onto $I = [-\frac{\sigma}{2}, \frac{\sigma}{2}]$. In the task descriptions below, $x \in J$ denotes the input before normalization, and $\widetilde{f}(x)$ includes input normalization. We also specify the approximation-error bound η_{app} used in the convex optimization.

For sigmoid and tanh, we compare with Bian et al.'s method [6]. For sigmoid, we set $J = [-8, 8]$ as in [6]. For tanh, we set $J = [-16, 16]$ as in [39]. We set $\eta(x) = 2^{-12}$ for both tasks. Bian et al. report a degree-34 polynomial for high-precision sigmoid evaluation. For our error target, we use their Fourier-extension algorithm to construct a degree-14 polynomial for sigmoid and a degree-63 polynomial for tanh. For SWIFT, we obtain the trigonometric polynomials using our convex optimization formulation with $\eta_{\mathrm{app}} = 10^{-6}$ for sigmoid and $\eta_{\mathrm{app}} = 7 \times 10^{-5}$ for tanh.

For ReLU, we compare with Lee et al.'s method [26]. We use $J = [-1, 1]$ and $\eta(x) = 2^{-12}$ to bound the output error. Lee et al. [26]'s method evaluates ReLU as $x(1 + \mathsf{Sign}(x))/2$ by approximating Sign with a composition of polynomials of degrees $\{15, 15, 27\}$, with additional bootstrapping between $\{15, 15\}$ and $\{27\}$. We do not adopt RBoot [45] as a baseline, since it gives higher latency and larger error than [26], under our implementation parameters. For SWIFT, we obtain the trigonometric polynomial using our convex optimization formulation with $\eta_{\mathrm{app}} = 9 \times 10^{-5}$.

For $1/x$ and $1/\sqrt{x}$, we compare with Rho et al. [39]. We use $J = [0.04, 1]$ and a degree-63 Chebyshev approximation evaluated by the PS algorithm for $1/x$, as the baseline in [39]. For $1/\sqrt{x}$, we use $J = [0.0005, 1]$. Rho et al.'s method [39] first coarsely approximates $1/\sqrt{x}$ with a degree-127 Chebyshev polynomial, then uses three Newton-Raphson steps. Between the Chebyshev polynomial evaluation and the Newton-Raphson steps, one additional bootstrapping is applied. For

both methods, since the target functions grow as $x \to 0^+$, we control relative error by setting $\eta(x) = 2^{-12}/x$ for $1/x$ and $\eta(x) = 2^{-12}/\sqrt{x}$ for $1/\sqrt{x}$. Thus, each computed output y satisfies $|xy-1| < 2^{-12}$ or $|\sqrt{x}\,y-1| < 2^{-12}$, respectively. For SWIFT, we obtain the trigonometric polynomials using our convex optimization formulation with relative approximation-error bounds $\eta_{\text{app}} = 1.2 \times 10^{-4}$ for $1/x$ and $\eta_{\text{app}} = 7 \times 10^{-5}$ for $1/\sqrt{x}$.

For the CKKS baselines, we use $\log_2 N = 15$ for 4-to-4 LUT and sigmoid, and $\log_2 N = 16$ for the other tasks. For SWIFT, we use $\log_2 N = 15$ for all tasks. For each continuous task, we choose a trigonometric polynomial f following Section 5.1, and specify its degree d, coefficient norm $\|c\|_1$, and interval length σ. Table 4 lists the task specifications.

Task	Goal for each Task			Trigonometric polynomial for SWIFT			Comparing Baselines	
	J Domain	η Target error	e_{pert} Input pert.	d degree	$\|c\|_1$ Coeff. Norm	σ Length	Method (CKKS/TFHE)	CKKS depth
4-to-4 LUT	$2^{-4}\mathbb{Z}/\mathbb{Z}$	$< 2^{-5}$	2^{-8}	$2^4 - 1$	0.491	1	[2]/ [12, 27]	5
8-to-8 LUT	$2^{-8}\mathbb{Z}/\mathbb{Z}$	$< 2^{-9}$	2^{-16}	$2^8 - 1$	2.059	1	[2]/ [12, 27]	9
12-to-12 LUT	$2^{-12}\mathbb{Z}/\mathbb{Z}$	$< 2^{-13}$	2^{-24}	$2^{12} - 1$	8.208	1	[16]/ [41]	6
sigmoid	$[-8, 8]$	2^{-12}	0	2^4	0.352	0.625	[6]	4
tanh	$[-16, 16]$	2^{-12}	0	2^6	1.040	0.88	[6]	6
ReLU	$[-1, 1]$	2^{-12}	0	2^9	0.705	0.98125	[26]	$8 + 6^\dagger$
$1/x$	$[0.04, 1]$	2^{-12^*}	0	2^5	24.773	0.85	[39]	6
$1/\sqrt{x}$	$[0.0005, 1]$	2^{-12^*}	0	2^{11}	33.484	0.9	[39]	$7 + 6^\dagger$

Table 4: Experimental tasks and trigonometric-polynomial parameters used by SWIFT. Each continuous domain J is affinely mapped onto $I = [-\frac{\sigma}{2}, \frac{\sigma}{2}] \subset \mathbb{R}/\mathbb{Z}$ for $\sigma < 1$. *For singular functions, we use the relative error. †The multiplicative depths before and after an intermediate bootstrap. Multiplicative depth refers to the prior CKKS method.

6.2 Experimental Results

Our implementation uses CryptoLab's C++ HEaveN library on an Intel Xeon Gold 6542Y CPU with one CPU thread. We reimplement all CKKS-based methods in the same library. For BatchBoot and Tetris, we estimate their latencies from the published results, adjusted by the ratio of PassMark single-thread scores between our CPU and the CPUs used in their experiments [35–37]. Each measured latency is the mean of ten runs and includes the initial bootstrap, polynomial evaluation, and any additional bootstrapping during evaluation. We report the maximum error over all test inputs and executions.

Parameters. For all SWIFT parameters and the CKKS baselines for 4-to-4 LUT and sigmoid, we fix $(h, \widetilde{h}) = (256, 32)$, $\log_2 N = 15$, and $\log_2 PQ \leq 833$. For the

4-to-4 LUT	L	β	L/β	Prec.
$t=1$	1.972	1024	1.926	7.21
$t=2$	1.990	2048	0.9715	7.21
$t=3$	2.166	2730	0.7935	7.21
$t=4$	3.659	2048	1.787	7.21
Prior	2.884	16384	0.1760	7.20

8-to-8 LUT	L	β	L/β	Prec.
$t=1$	1.952	64	30.50	14.99
$t=2$	2.235	512	4.365	14.60
$t=3$	2.503	1365	1.834	14.51
$t=4$	7.941	1024	7.755	14.71
Prior	9.981	32768	0.3046	13.03

12-to-12 LUT	L	β	L/β	Prec.
$t=1$	2.104	4	526.1	15.79
$t=2$	2.973	128	23.23	15.10
$t=3$	5.023	341	14.73	13.73
$t=4$	26.485	512	51.73	13.77
Prior	47.955	32768	1.463	13.84

sigmoid	L	β	L/β	Prec.
$t=1$	1.867	1024	1.823	19.26
$t=2$	2.016	2048	0.9843	17.94
$t=3$	2.149	2730	0.7872	17.16
$t=4$	3.571	2048	1.744	19.04
Prior	2.457	16384	0.1500	13.37

tanh	L	β	L/β	Prec.
$t=1$	1.922	256	7.509	13.78
$t=2$	2.091	1024	2.042	13.64
$t=3$	2.492	1365	1.826	13.47
$t=4$	3.562	2048	1.739	13.76
Prior	7.090	32768	0.2164	12.68

ReLU	L	β	L/β	Prec.
$t=1$	2.021	32	63.16	13.43
$t=2$	2.215	512	4.326	13.38
$t=3$	3.194	682	4.683	13.27
$t=4$	7.883	1024	7.699	13.42
Prior	10.766	32768	0.3286	13.02

$1/x$	L	β	L/β	Prec.*
$t=1$	1.925	512	3.760	12.95
$t=2$	2.014	2048	0.9834	12.70
$t=3$	2.192	2730	0.8029	12.46
$t=4$	3.610	2048	1.763	12.97
Prior	5.177	32768	0.1580	13.48

$1/\sqrt{x}$	L	β	L/β	Prec.*
$t=1$	2.082	8	260.2	13.28
$t=2$	2.405	256	9.395	13.08
$t=3$	3.188	682	4.674	13.24
$t=4$	7.891	1024	7.707	13.47
Prior	11.211	32768	0.3421	12.07

Table 5: Latency L (s), batch size β, amortized time L/β (ms/input), and precision (bits) for the tasks in Table 4. *Relative-error precision.

other CKKS baselines, we use $(h, \widetilde{h}) = (192, 32)$, $\log_2 N = 16$, and $\log_2 PQ \leq 1552$. We use these modulus bounds for a target of 128-bit security, following estimates from the Lattice Estimator [1]. Here, $\widetilde{h}$ is the sparse-key Hamming weight used during bootstrapping [7]. The detailed CKKS parameters are listed in Table 6 in Appendix A.

Experiment 1: SWIFT with Varying *t*. First, we compare the latencies and amortized times of SWIFT for $t = 1, 2, 3, 4$ on each task. Table 5 summarizes the results. We also include the CKKS baseline implementation results for reference. All measured errors meet the error targets in Table 4.

Among the eight tasks with existing measurements, $t = 1$ gives the lowest latency, whereas $t = 3$ gives the lowest amortized time for six tasks. For ReLU, $t = 2$ gives the lowest amortized time. For large degrees, however, $t = 1$ accommodates few inputs: only 4 for 12-to-12 LUT and 8 for $1/\sqrt{x}$. Thus, $t = 2$ or 3 is useful when larger batches are required. For the tested tasks other than tanh, $t = 4$ provides no improvement over $t = 3$ in either latency or amortized time.

The batch size β does not always increase with t. For the low-degree tasks, including 4-to-4 LUT and sigmoid, increasing t from 3 to 4 reduces β from 2730 to 2048. In these cases, the factor size remains $n = 2$, so the additional factor reduces the batch size. For 12-to-12 LUT, whose polynomial has degree $d = 4095$, the same change in t increases β from 341 to 512. The increase in batch size, however, comes with a higher evaluation cost. For 12-to-12 LUT, increasing t from 3 to 4 raises the latency from 5.023 to 26.485 seconds, while the batch size increases by only about 50%. Consequently, amortized time increases from 14.73 to 51.73 ms/input. This behavior is consistent with the higher degree dependence of the key-switching and plaintext-multiplication counts, together with the additional multiplicative depth, shown in Table 1.

For $t = 3$, ReLU and $1/\sqrt{x}$ have the same batch size $\beta = 682$, despite their different degrees 2^9 and 2^{11}, respectively. Since the factor size n is restricted to powers of two, both degrees require $n = 8$, giving a degree capacity of $(2n)^3/2 = 2^{11}$. The ReLU coefficients are therefore zero-padded to the same tensor dimensions used for $1/\sqrt{x}$.

Experiment 2: Latency across Batch Sizes. Next, we compare the total latency $L(\beta)$ for each requested batch size $\beta \in \{2^m : m = 0, \ldots, 15\}$. For each method, let $L_{\text{meas}}(\beta)$ denote the mean total latency measured for a batch of β inputs (or the estimated latency). For each requested β, we take $L(\beta) = \min_{B \geq \beta} L_{\text{meas}}(B)$, since a run measured for B inputs can leave $B - \beta$ inputs unused.

Optimizations. The latency of a CKKS-based method can be reduced according to the number of active slots. In particular, the costs of CtS and StC decrease when fewer values are packed. For each requested batch size β, we therefore adapt the bootstrapping linear transforms to the corresponding number of slots and measure the latency. We also optimize SWIFT with various batch sizes. For smaller batch sizes, we used fewer-slot bootstrapping as above, and for larger batch sizes we applied the batch-CKKS bootstrapping method of Bae et al. [4]. This method shares computations in StC and CtS across ciphertexts to reduce latency. A known technique packs N inputs into the real and imaginary parts of the slots, shares StC and CtS, and then applies EvalMod or EvalExp separately. Since we compare latency at the requested batch sizes rather than full-packing throughput, we generally do not use this optimization. The exceptions are the 4-to-4 LUT and sigmoid baselines with $\log_2 N = 15$ at batch size 2^{15}, for which we report the results with this optimization.

Figure 4 shows the results. The shaded regions indicate the tested batch sizes for which SWIFT achieves the lowest total latency. For all continuous-domain tasks, SWIFT reduces latency over the CKKS baselines at every tested batch size up to 1024. For larger batches, the latency of SWIFT increases, eventually exceeding that of the CKKS baselines.

For the lookup-table tasks, SWIFT achieves the lowest latency for the tested batch sizes from 1024 to 2048 for 4-to-4 LUT, from 32 to 4096 for 8-to-8 LUT, and

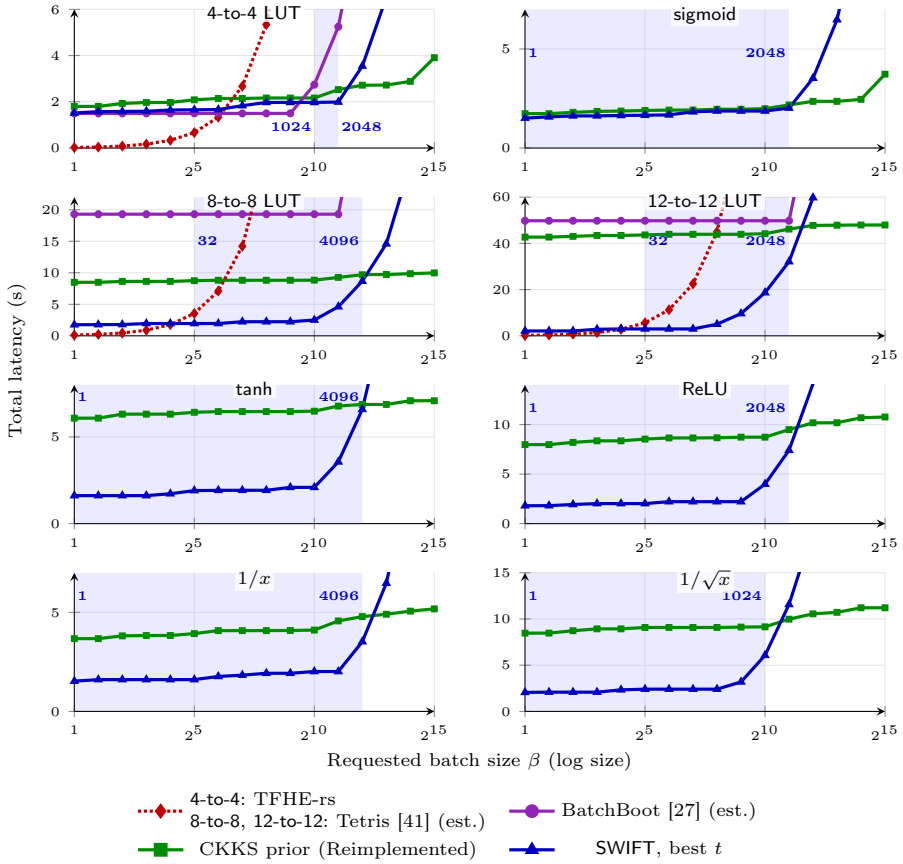


Fig. 4: Total latency as a function of the requested batch size β. Detailed values are given in Appendix A.

from 32 to 2048 for 12-to-12 LUT. The gain is particularly large for 12-to-12 LUT. At a batch size of 256, SWIFT takes 5.02 seconds, compared with 43.92 seconds for the fastest baseline, giving a speedup of 8.75×. In addition, for both 8-to-8 LUT and 12-to-12 LUT, SWIFT outperforms the estimated DM/CGGI-based baselines at every tested batch size of 32 or more.

For 4-to-4 LUT at batch sizes of at most 512, the latency of SWIFT is comparable to, but generally slightly higher than, the estimated latency of BatchBoot. We use SWIFT parameters shared across tasks and chosen to support high-degree polynomial evaluation. Optimizing these parameters specifically for 4-to-4 LUT may further reduce the latency.

For sigmoid and 4-to-4 LUT, the CKKS baselines also operate at $\log_2 N = 15$, the same ring degree as SWIFT. The gains over these baselines are more modest. At these low polynomial degrees, bootstrapping accounts for most of the latency

of SWIFT. Thus, reducing the key-switching cost of polynomial evaluation has a limited effect on total latency when the ring degree remains unchanged.

7 Conclusion

We presented SWIFT, a CKKS functional bootstrapping method that reduces latency for moderate batch sizes. SWIFT packs integer multiples of each input into unused SIMD slots and applies ExpBTS to generate exponential powers in parallel. It then evaluates the polynomial through a tensor contraction, reducing both multiplicative depth and the asymptotic key-switching count compared with Paterson–Stockmeyer evaluation. The reduced depth lowers the required ciphertext modulus, allowing a smaller ring degree at the same security level.

We also derived numerical-error bounds for SWIFT that grow linearly with the coefficient norm $\|c\|_1$. Motivated by these bounds, we formulated a convex optimization problem that minimizes the coefficient norm for each degree d, subject to the prescribed approximation-error bound. This approach yields trigonometric approximations for functions such as $1/x$ and $1/\sqrt{x}$, trading higher polynomial degrees for smaller coefficient norms. The computational cost of these higher degrees is mitigated by the low-depth evaluation of SWIFT.

At a batch size of 32, our implementation achieves speedups of 4.48× and 14.68× over the corresponding CKKS baselines for 8-to-8 LUT and 12-to-12 LUT, respectively, while also outperforming the estimated DM/CGGI-based baselines. Furthermore, SWIFT enables low-depth evaluation of singular functions such as $1/\sqrt{x}$ through trigonometric approximations, overcoming the difficulty caused by coefficient growth. For 512 inputs, our implementation evaluates $1/\sqrt{x}$ with 12-bit relative precision in 3.19 seconds, achieving a 2.86× speedup over the CKKS baseline.

Several limitations and directions for further study remain. First, the choice of factor sizes can be optimized for different polynomial degrees. Our implementation uses factor sets of equal size $n = 2^m$, corresponding to $d = \frac{(2n)^t}{2} = 2^{(m+1)t-1}$. For degrees that do not have this form, we use zero padding, which may waste SIMD slots and increase the operation counts. Optimizing the packing and evaluation for unequal factor sizes or sizes that are not powers of two may reduce this overhead. Second, we have considered only univariate functions. Extending our construction to multivariate functions is also of interest. Third, adapting SWIFT to sparse frequency sets Λ is of interest, including those arising from sparse Hermite interpolation [3].

Acknowledgment

We thank Hanee Rhee for contributing to an earlier version of this work and Guillaume Hanrot for discussions on trigonometric approximation.

References

1. Albrecht, M.R., contributors: Security estimates for lattice problems. https://github.com/malb/lattice-estimator (2022)
2. Alexandru, A., Kim, A., Polyakov, Y.: General functional bootstrapping using ckks. In: Tauman Kalai, Y., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025. pp. 304–337. Cham: Springer Nature Switzerland (2025)
3. Alexandru, A., Kim, A., Polyakov, Y., Zheng, H.: Sparse hermite interpolation method for discrete-CKKS functional bootstrapping. Cryptology ePrint Archive, Paper 2026/1026 (2026), https://eprint.iacr.org/2026/1026
4. Bae, Y., Cheon, J.H., Hanrot, G., Park, J.H., Stehlé, D.: Plaintext-ciphertext matrix multiplication and fhe bootstrapping: Fast and fused. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024. pp. 387–421. Cham: Springer Nature Switzerland (2024)
5. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with ckks. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024. pp. 330–360. Springer Nature Singapore, Singapore (2025)
6. Bian, S., Fu, Y., Shen, R., Pan, H., Wang, A., Guan, Z.: High-precision functional bootstrapping for CKKS from fourier extension. In: Advances in Cryptology – EUROCRYPT 2026. Lecture Notes in Computer Science, vol. 16544, pp. 274–303. Springer Nature Switzerland (2026). https://doi.org/10.1007/978-3-032-25327-9_10, https://link.springer.com/chapter/10.1007/978-3-032-25327-9_10
7. Bossuat, J.P., Troncoso-Pastoriza, J.R., Hubaux, J.P.: Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. Cryptology ePrint Archive, Paper 2022/024 (2022), https://eprint.iacr.org/2022/024
8. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference. p. 309–325. ITCS '12, Association for Computing Machinery, New York, NY, USA (2012)
9. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018. pp. 360–384. Cham: Springer International Publishing (2018)
10. Cheon, J.H., Hanrot, G., Kim, J., Stehlé, D.: Ship: A shallow and highly parallelizable ckks bootstrapping algorithm. In: Advances in Cryptology – EUROCRYPT 2025: 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4–8, 2025, Proceedings, Part III. p. 398–428. Springer-Verlag, Berlin, Heidelberg (2025). https://doi.org/10.1007/978-3-031-91131-6_14, https://doi.org/10.1007/978-3-031-91131-6_14
11. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) Advances in Cryptology – ASIACRYPT 2017. pp. 409–437. Cham: Springer International Publishing (2017)
12. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. J. Cryptol. **33**(1), 34–91 (Jan 2020)
13. Coron, J.S., Köstler, R.: Low-latency bootstrapping for CKKS using roots of unity. Cryptology ePrint Archive, Paper 2025/651 (2025), https://eprint.iacr.org/2025/651

14. De Micheli, G., Kim, D., Micciancio, D., Suhl, A.: Faster amortized FHEW bootstrapping using ring automorphisms. In: Public-Key Cryptography – PKC 2024. Lecture Notes in Computer Science, vol. 14604, pp. 322–353. Springer (2024). https://doi.org/10.1007/978-3-031-57728-4_11, https://doi.org/10.1007/978-3-031-57728-4_11
15. Ducas, L., Micciancio, D.: Fhew: Bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology – EUROCRYPT 2015. pp. 617–640. Springer Berlin Heidelberg, Berlin, Heidelberg (2015)
16. Dumezy, J., Alexandru, A., Polyakov, Y., Clet, P.E., Chakraborty, O., Boudguiga, A.: Evaluating larger lookup tables using ckks. IACR Transactions on Cryptographic Hardware and Embedded Systems **2026**(1), 559–591 (Jan 2026). https://doi.org/10.46586/tches.v2026.i1.559-591, https://tches.iacr.org/index.php/TCHES/article/view/12693
17. Dumezy, J., Suvanto, E.: Character block encodings for discrete CKKS: Single-level LUTs and low-depth arithmetic. Cryptology ePrint Archive, Paper 2026/1200 (2026), https://eprint.iacr.org/2026/1200
18. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144 (2012)
19. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing. p. 169–178. STOC, Association for Computing Machinery (2009)
20. Goulart, P.J., Chen, Y.: Clarabel: An interior-point solver for conic programs with quadratic objectives. Mathematical Programming Computation (2026). https://doi.org/10.1007/s12532-026-00320-7, https://doi.org/10.1007/s12532-026-00320-7
21. Guimarães, A., Pereira, H.V.L.: Fast amortized bootstrapping with small keys and polynomial noise overhead. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. pp. 2967–2981. Association for Computing Machinery (2025). https://doi.org/10.1145/3719027.3765181, https://doi.org/10.1145/3719027.3765181
22. Guimarães, A., Pereira, H.V.L., van Leeuwen, B.: Amortized bootstrapping revisited: Simpler, asymptotically-faster, implemented. In: Advances in Cryptology – ASIACRYPT 2023. Lecture Notes in Computer Science, vol. 14443, pp. 3–35. Springer (2023). https://doi.org/10.1007/978-981-99-8736-8_1, https://doi.org/10.1007/978-981-99-8736-8_1
23. Guo, H., Tang, R., Ye, Y., Li, Z., He, X.: Deepfm: A factorization-machine based neural network for ctr prediction. In: Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence. pp. 1725–1731 (2017)
24. Halevi, S., Shoup, V.: Algorithms in HElib. Cryptology ePrint Archive, Paper 2014/106 (2014)
25. Kim, J., Cheon, J.H., Yeo, Y.: OverModRaise: Reducing modulus consumption of CKKS bootstrapping. IACR Communications in Cryptology (2025). https://doi.org/10.62056/a3n5qjp10, https://doi.org/10.62056/a3n5qjp10
26. Lee, J., Lee, E., Lee, J.W., Kim, Y., Kim, Y.S., No, J.S.: Precise approximation of convolutional neural networks for homomorphically encrypted data. IEEE Access **11**, 62062–62076 (2023). https://doi.org/10.1109/ACCESS.2023.3287564
27. Li, Z., Wang, H., Zhao, Y., Li, L., Wang, Z., He, J., Wei, C., Yan, Y., Guo, L.: BatchBoot: Fast batched bootstrapping for TFHE scheme and practical applications. In: 35th USENIX Security Symposium (USENIX Security 26) (2026), https://www.usenix.org/conference/usenixsecurity26/presentation/li-zhihao

28. Lobo, M.S., Vandenberghe, L., Boyd, S., Lebret, H.: Applications of second-order cone programming. Linear Algebra and its Applications **284**(1), 193–228 (1998). https://doi.org/https://doi.org/10.1016/S0024-3795(98)10032-0, https://www.sciencedirect.com/science/article/pii/S0024379598100320, international Linear Algebra Society (ILAS) Symposium on Fast Algorithms for Control, Signals and Image Processing
29. Ma, S., Huang, T., Wang, A., Xu, C., Wei, T., Wang, X.: Meta-PBS: Compact high-precision programmable bootstrapping. Cryptology ePrint Archive, Paper 2025/2284 (2025), https://eprint.iacr.org/2025/2284
30. Micciancio, D., Sorrell, J.: Ring packing and amortized FHEW bootstrapping. In: 45th International Colloquium on Automata, Languages, and Programming (ICALP 2018). Leibniz International Proceedings in Informatics (LIPIcs), vol. 107, pp. 100:1–100:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2018). https://doi.org/10.4230/LIPIcs.ICALP.2018.100, https://doi.org/10.4230/LIPIcs.ICALP.2018.100
31. Naumov, M., Mudigere, D., Shi, H.J.M., Huang, J., Sundaraman, N., Park, J., Wang, X., Gupta, U., Wu, C.J., Azzolini, A.G., Dzhulgakov, D., Mallevich, A., Cherniavskii, I., Lu, Y., Krishnamoorthi, R., Yu, A., Kondratenko, V., Pereira, S., Chen, X., Chen, W., Rao, V., Jia, B., Xiong, L., Smelyanskiy, M.: Deep learning recommendation model for personalization and recommendation systems. arXiv preprint arXiv:1906.00091 (2019)
32. Okada, H., Player, R., Pohmann, S.: Homomorphic polynomial evaluation using Galois structure and applications to BFV bootstrapping. In: Advances in Cryptology – ASIACRYPT 2023. Lecture Notes in Computer Science, vol. 14443, pp. 69–100. Springer (2023). https://doi.org/10.1007/978-981-99-8736-8_3, https://eprint.iacr.org/2023/1304
33. Ouyang, R., Liu, J., Gao, P., Xiao, P., Zhang, H., Au, M.H., Zhang, C., Ren, K., Chen, C.: CKKS bootstrapping in less than 10ms. Cryptology ePrint Archive, Paper 2026/2119 (2026), https://eprint.iacr.org/2026/2119
34. Park, J., Park, S., Park, J.H., Ahn, J.H., Cheon, J.H., Hanrot, G., Kim, J.W., Park, M., Stehlé, D.: Scaling up FHE-based privacy-preserving ML: Higher throughput, longer inputs for Llama-3-8b. arXiv preprint arXiv:2601.18511 (2026), https://arxiv.org/abs/2601.18511
35. PassMark Software: Intel core i7-13700 benchmark (2026), https://www.cpubenchmark.net/cpu.php?cpu=Intel+Core+i7-13700&id=4992, accessed September 2, 2026
36. PassMark Software: Intel xeon gold 6258r benchmark (2026), https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Gold+6258R+%40+2.70GHz&id=3860, accessed September 10, 2026
37. PassMark Software: Intel xeon gold 6542y benchmark (2026), https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Gold+6542Y&id=6252, accessed September 2, 2026
38. Paterson, M.S., Stockmeyer, L.J.: On the number of nonscalar multiplications necessary to evaluate polynomials. SIAM Journal on Computing **2**(1), 60–66 (1973). https://doi.org/10.1137/0202007
39. Rho, D., Kim, T., Park, M., Kim, J.W., Chae, H., Ryu, E.K., Cheon, J.H.: Encryption-friendly llm architecture. In: International Conference on Learning Representations (2025), https://proceedings.iclr.cc/paper_files/paper/2025/hash/6715b4e97be055687c1ecaf33913d358-Abstract-Conference.html

40. Rivest, R.L., Adleman, L.M., Dertouzos, M.L.: On data banks and privacy homomorphisms. In: Foundations of Secure Computation. pp. 169–180. Academic Press (1978)
41. Wang, R., Bai, J., Shen, X., Lu, X., Li, Z., Xiang, B., Wang, Z., Wang, H., Zhao, L., Wang, K., Hou, R.: Tetris: Versatile TFHE LUT and its application to FHE instruction set architecture. Cryptology ePrint Archive, Paper 2025/1623 (2025), https://eprint.iacr.org/2025/1623
42. Wang, R., Ha, J., Shen, X., Lu, X., Chen, C., Wang, K., Lee, J.: Refined TFHE leveled homomorphic evaluation and its application. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. pp. 2309–2323. Association for Computing Machinery (2025). https://doi.org/10.1145/3719027.3744873
43. Xiao, P., Ouyang, R., Zhang, H., Zhang, J., Liu, J.: Bootstrapping is all you need: Secure transformer inference via improved CKKS functional bootstrapping. Cryptology ePrint Archive, Paper 2026/1255 (2026), https://eprint.iacr.org/2026/1255
44. Yalçın, E., Tanyıldız, H., Aslan, S., Demir, S.C., Sucu, M., Uzay, F.İ., Aykut, S., Yalçın, E.M., Biçer, A.: Explainable tabular deep learning models for antenatal cesarean delivery prediction in multiparous women. BMC Pregnancy and Childbirth **26**(453) (2026). https://doi.org/10.1186/s12884-026-08934-4
45. Yang, Z., Niu, C., Wei, B., Huang, Z., Hong, C., Wei, T.: RBOOT: Accelerating homomorphic neural network inference by fusing ReLU within bootstrapping. Cryptology ePrint Archive, Paper 2025/1534 (2025), https://eprint.iacr.org/2025/1534

Use of AI Tools.

We used OpenAI's ChatGPT and Codex to assist with drafting and revising the manuscript, developing and debugging experimental code, and preparing tables and figures. The authors take full responsibility for the paper's content and reported results.

A Implementation Details

A.1 HE Parameters

Table 6 lists the CKKS parameters selected for the eight tasks. We first selected the multiplication primes and then chose the bootstrapping parameters accordingly. The entries under $\log_2 P$ describe the auxiliary primes selected automatically by HEaveN. For each task, we selected the parameters that yielded the lowest latency among those tested, although these parameters may not be optimal.

Method	$\log_2 N$	Prime count × size (bits)						$\log_2(PQ_L)$
		$\log_2 P$	CtS*	EvalExp/ EvalMod	Mult	StC	q_0	
SWIFT ($t = 1$)	15	$2 \times 61 + 1 \times 56$	$2(+1) \times 33$	8×48	1×44	3×39	1×44	833
SWIFT ($t = 2, 3$)	15	$2 \times 61 + 1 \times 29$	$2(+1) \times 32$	8×46	2×44	3×34	1×44	817
SWIFT ($t = 4$)	15	$2 \times 61 + 1 \times 23$	$2(+1) \times 37$	8×46	3×44	2×35	1×44	833
4-to-4 LUT	15	$1 \times 61 + 1 \times 58$	$2(+1) \times 31$	8×45	5×31	3×34	1×31	829
sigmoid	15	$1 \times 61 + 1 \times 40$ $+1 \times 41$	$2(+1) \times 32$	8×46	4×31	3×34	1×32	832
8-to-8 LUT	16	6×61	$2(+1) \times 48$	8×60	9×38	3×38	1×38	1436
12-to-12 LUT	16	5×61	$2(+1) \times 47$	8×60	6×48	3×48	1×61	1372
tanh	16	7×61	$2(+1) \times 38$	8×49	6×32	3×36	1×33	1228
ReLU	16	8×61	$2(+1) \times 38$	8×49	8×38	3×38	1×48	1422
$1/x$	16	8×61	$2(+1) \times 44$	8×52	6×34	3×38	1×44	1354
$1/\sqrt{x}$	16	9×61	$2(+1) \times 48$	8×60	7×40	3×31	1×50	1548

Table 6: HE parameters for SWIFT and the prior CKKS methods. We use $(h, \widetilde{h}) = (256, 32)$ for $\log_2 N = 15$ and $(192, 32)$ for $\log_2 N = 16$. * The first CtS layer is fused with ModRaise, following [25]. This layer performs rescaling before key switching, so no key-switching key is required at its largest input modulus. Hence, only the two rescaling primes for the remaining CtS layers contribute to $\log_2(PQ_L)$.

A.2 Latencies for Varying Batch Sizes

Table 7 gives the SWIFT latencies for each t, and Table 8 gives the baseline latencies. For each fixed t, we use the minimum latency over the tested slot counts and batching choices.

Table 7: Total latencies (s) of SWIFT for each t and requested batch size β. Bold values indicate the minimum over t for each task and β.

	4-to-4 LUT				8-to-8 LUT			
β	$t=1$	$t=2$	$t=3$	$t=4$	$t=1$	$t=2$	$t=3$	$t=4$
1	**1.5133**	1.7186	1.8684	3.0814	**1.7577**	1.9982	2.1998	7.4808
2	**1.5754**	1.7186	1.8684	3.0814	**1.7735**	2.0470	2.2617	7.4808
4	**1.5827**	1.7796	1.8684	3.1509	**1.7873**	2.0470	2.3158	7.5227
8	**1.5924**	1.8079	1.9362	3.2244	**1.9520**	2.0553	2.3366	7.5428
16	**1.6447**	1.8079	1.9362	3.2244	**1.9520**	2.0553	2.3366	7.5532
32	**1.6553**	1.8139	1.9765	3.2244	**1.9520**	2.0566	2.3366	7.5532
64	**1.6661**	1.8289	1.9765	3.2458	**1.9520**	2.2350	2.3366	7.5532
128	1.8456	**1.8319**	1.9765	3.2458	3.5377	**2.2350**	2.5032	7.7761
256	**1.9724**	1.9896	2.1664	3.4639	6.4230	**2.2350**	2.5032	7.9410
512	**1.9724**	1.9896	2.1664	3.5988	12.2992	**2.2350**	2.5032	7.9410
1024	**1.9724**	1.9896	2.1664	3.6590	24.5984	3.9930	**2.5032**	7.9410
2048	3.3016	**1.9896**	2.1664	3.6590	49.1967	7.4170	**4.6198**	15.4278
4096	5.9508	**3.5446**	3.8857	6.6655	98.3934	14.1432	**8.6747**	30.1528
8192	11.3548	**6.5203**	7.2067	12.6282	196.7869	28.2865	**14.5971**	59.3976
16384	22.7097	12.3499	**12.0280**	24.3484	393.5737	56.5730	**27.2904**	118.7952
32768	45.4193	24.6997	**22.5193**	48.6969	787.1474	113.1459	**52.4757**	237.5903

	12-to-12 LUT				sigmoid			
β	$t=1$	$t=2$	$t=3$	$t=4$	$t=1$	$t=2$	$t=3$	$t=4$
1	**2.1043**	2.6937	4.7146	25.8633	**1.5090**	1.7036	1.8161	3.1144
2	**2.1043**	2.6937	4.7146	25.8633	**1.5723**	1.7195	1.8161	3.1168
4	**2.1043**	2.8314	4.7146	25.8633	**1.6219**	1.7586	1.9100	3.1486
8	3.7631	**2.8326**	4.7146	25.8633	**1.6219**	1.8032	1.9100	3.1872
16	6.8738	**2.9731**	4.7236	25.8993	**1.6428**	1.8032	1.9100	3.2168
32	13.2009	**2.9731**	5.0227	25.8993	**1.6553**	1.8275	1.9917	3.2168
64	26.4017	**2.9731**	5.0227	26.1639	**1.6726**	1.8275	1.9917	3.2168
128	52.8035	**2.9731**	5.0227	26.2854	1.8535	**1.8286**	1.9917	3.2168
256	105.6069	5.4952	**5.0227**	26.4853	**1.8671**	1.9972	2.1489	3.4670
512	211.2139	10.4215	**9.6173**	26.4853	**1.8671**	2.0159	2.1489	3.5714
1024	422.4277	20.1522	**18.6697**	52.3170	**1.8671**	2.0159	2.1489	3.5714
2048	844.8555	40.3045	**32.0883**	103.9312	3.3004	**2.0159**	2.1489	3.5714
4096	1689.7109	80.6090	**59.7741**	206.9545	5.9484	**3.5236**	3.8674	6.5868
8192	3379.4218	161.2180	**114.9443**	413.9089	11.3500	**6.4783**	7.1700	12.4707
16384	6758.8436	322.4359	**224.8601**	827.8179	22.7000	12.2657	**11.9638**	24.0335
32768	13517.6873	644.8719	**444.6916**	1655.6358	45.4000	24.5315	**22.4000**	48.0671

	tanh				ReLU			
β	$t=1$	$t=2$	$t=3$	$t=4$	$t=1$	$t=2$	$t=3$	$t=4$
1	**1.6145**	1.7974	2.2182	3.1117	**1.8038**	2.0076	2.9758	7.5019
2	**1.6145**	1.8558	2.2182	3.1179	**1.8133**	2.0454	3.0088	7.5096
4	**1.6145**	1.8960	2.2927	3.1508	**1.9303**	2.0454	3.0463	7.5096
8	**1.6145**	1.8960	2.2927	3.1912	**2.0211**	2.0504	3.0463	7.5096
16	**1.7222**	1.8960	2.3183	3.2111	**2.0211**	2.0504	3.0823	7.5096
32	1.9088	**1.9048**	2.3274	3.2111	**2.0211**	2.0556	3.0823	7.5096
64	1.9222	**1.9197**	2.3558	3.2194	3.5916	**2.2150**	3.1936	7.5096
128	**1.9222**	2.0912	2.4922	3.2194	6.5309	**2.2150**	3.1936	7.7544
256	**1.9222**	2.0912	2.4922	3.4634	12.5150	**2.2150**	3.1936	7.8834
512	3.4167	**2.0912**	2.4922	3.5617	25.0300	**2.2150**	3.1936	7.8834
1024	6.1809	**2.0912**	2.4922	3.5617	50.0600	**3.9875**	6.0801	7.8834
2048	11.8150	3.7016	4.5788	**3.5617**	100.1199	**7.4061**	11.5953	15.3185
4096	23.6301	6.8343	8.5928	**6.5916**	200.2398	**14.1214**	19.7081	29.9343
8192	47.2602	12.9778	14.4538	**12.4804**	400.4797	**28.2427**	36.7823	58.9607
16384	94.5204	25.9556	27.0244	**24.0529**	800.9594	**56.4855**	70.7293	117.9214
32768	189.0408	51.9112	51.9641	**48.1058**	1601.9187	**112.9710**	138.1986	235.8428

	$1/x$				$1/\sqrt{x}$			
β	$t=1$	$t=2$	$t=3$	$t=4$	$t=1$	$t=2$	$t=3$	$t=4$
1	**1.5271**	1.7234	1.8535	3.0916	**2.0588**	2.2982	2.9828	7.4239
2	**1.6045**	1.7234	1.8535	3.0916	**2.0816**	2.2982	3.0441	7.4686
4	**1.6045**	1.7642	1.9324	3.1642	**2.0816**	2.2982	3.0441	7.4973
8	**1.6045**	1.7642	1.9806	3.1879	**2.0816**	2.3274	3.1133	7.4973
16	**1.6045**	1.7642	1.9806	3.1879	3.7067	**2.3332**	3.1157	7.5353
32	**1.6045**	1.7642	1.9927	3.2146	6.7611	**2.4051**	3.1343	7.6137
64	1.8796	**1.7642**	1.9985	3.2360	12.9754	**2.4051**	3.1878	7.6137
128	1.9249	**1.8303**	2.0003	3.2360	25.9509	**2.4051**	3.1878	7.7451
256	**1.9249**	2.0106	2.1758	3.4828	51.9017	**2.4051**	3.1878	7.8915
512	**1.9249**	2.0140	2.1919	3.6097	103.8035	4.5041	**3.1878**	7.8915
1024	3.3603	**2.0140**	2.1919	3.6097	207.6070	8.4394	**6.0675**	7.8915
2048	6.0683	**2.0140**	2.1919	3.6097	415.2140	16.1879	**11.5702**	15.3390
4096	11.5898	**3.5210**	3.8671	6.6034	830.4280	32.3758	**19.6643**	29.9752
8192	23.1796	**6.4731**	7.1693	12.5041	1660.8559	64.7516	**36.7008**	59.0425
16384	46.3591	12.2553	**11.9626**	24.1003	3321.7118	129.5033	**70.5727**	118.0850
32768	92.7182	24.5107	**22.3978**	48.2005	6643.4236	259.0066	**137.8917**	236.1699

	CKKS prior (reimplemented)								TFHE-rs	Tetris		BatchBoot		
β	4-to-4	8-to-8	12-to-12	sigmoid	tanh	ReLU	$1/x$	$1/\sqrt{x}$	4-to-4	8-to-8	12-to-12	4-to-4	8-to-8	12-to-12
1	1.8007	8.4904	42.7144	1.7366	6.0833	7.9770	3.6735	8.4697	0.0209	0.1111	0.1758	1.4983	19.2977	49.7929
2	1.8007	8.4904	42.7144	1.7366	6.0833	7.9770	3.6735	8.4697	0.0417	0.2221	0.3516	1.4983	19.2977	49.7929
4	1.9276	8.6330	43.0362	1.8009	6.3171	8.2005	3.8145	8.7282	0.0835	0.4443	0.7033	1.4983	19.2977	49.7929
8	1.9707	8.6330	43.4225	1.8486	6.3171	8.3568	3.8318	8.9306	0.1669	0.8885	1.4066	1.4983	19.2977	49.7929
16	1.9707	8.6330	43.4225	1.8665	6.3171	8.3568	3.8318	8.9306	0.3338	1.7771	2.8132	1.4983	19.2977	49.7929
32	2.0858	8.7530	43.6588	1.8960	6.4225	8.5353	3.9276	9.0800	0.6676	3.5542	5.6263	1.4983	19.2977	49.7929
64	2.1436	8.8116	43.9234	1.9155	6.4636	8.6510	4.0786	9.0800	1.3352	7.1084	11.2526	1.4983	19.2977	49.7929
128	2.1436	8.8116	43.9234	1.9155	6.4636	8.6510	4.0786	9.0800	2.6705	14.2168	22.5052	1.4983	19.2977	49.7929
256	2.1649	8.8116	43.9234	1.9554	6.4636	8.6654	4.0786	9.0800	5.3410	28.4336	45.0105	1.4983	19.2977	49.7929
512	2.1649	8.8116	43.9234	1.9554	6.4636	8.7273	4.0858	9.1238	10.6820	56.8671	90.0209	1.4983	19.2977	49.7929
1024	2.1649	8.8293	44.1767	1.9856	6.4867	8.7273	4.1065	9.1484	21.3640	113.7342	180.0418	2.7409	19.2977	49.7929
2048	2.5274	9.2662	46.1692	2.1745	6.7826	9.4939	4.5686	9.9795	42.7279	227.4684	360.0837	5.2475	19.2977	49.7929
4096	2.7173	9.6967	47.7717	2.3558	6.8746	10.1785	4.7856	10.5573	85.4559	454.9369	720.1674	10.4949	38.5953	99.5858
8192	2.7237	9.7281	47.8650	2.3558	6.8746	10.1870	4.9051	10.7151	170.9118	909.8737	1440.3348	20.9898	77.1907	199.1717
16384	2.8839	9.8554	47.9547	2.4571	7.0897	10.6894	5.0628	11.2105	341.8235	1819.7475	2880.6696	41.9796	154.3813	398.3434
32768	3.9089	9.9807	47.9547	3.7277	7.0897	10.7663	5.1768	11.2105	683.6471	3639.4949	5761.3392	83.9592	308.7627	796.6868

Table 8: Total latencies (s) of the baselines in Fig. 4. Tetris and BatchBoot values are estimates.

B Proof of the Factor-Set Construction

Proof of Proposition 1. Suppose that some n_i is even. Reorder the factors so that n_t is even, and define A_i by (6). Put $P_u := \prod_{i=1}^{u} 2n_i,\ P_0 := 1$. For $1 \le u < t$, we prove by induction that

$$\bigoplus_{i=1}^{u}(\pm A_i) = \left\{-\frac{n_t}{2}(P_u - 1) + n_t j : 0 \le j < P_u\right\},$$

and that every element has a unique representation as a sum. The case $u = 1$ follows from the definition of A_1. For $2 \le u < t$, write

$$s_j = -\frac{n_t}{2}(P_{u-1} - 1) + n_t j, \qquad 0 \le j < P_{u-1},$$
$$a_k = -\frac{n_t P_{u-1}}{2}(2n_u - 1) + n_t P_{u-1} k, \quad 0 \le k < 2n_u.$$

Then $\{a_k\} = \pm A_u$ and

$$s_j + a_k = -\frac{n_t}{2}(P_u - 1) + n_t(j + P_{u-1}k).$$

As (j, k) varies over the stated ranges, $j + P_{u-1}k$ takes each value in $\{0, \dots, P_u - 1\}$ once. This proves the induction step.

Since $d = n_t P_{t-1}$,

$$\bigoplus_{i<t}(\pm A_i) = \left\{-\frac{d - n_t}{2} + n_t j : 0 \le j < d/n_t\right\},$$

$$A_t - \frac{d - n_t}{2} + n_t j = \{n_t j + 1, \dots, n_t(j + 1)\}.$$

The sets in the second line partition $[d]$. Hence

$$\left(\bigoplus_{i<t}(\pm A_i)\right) \oplus A_t = [d], \qquad \left(\bigoplus_{i<t}(\pm A_i)\right) \oplus (-A_t) = -[d].$$

The sum $\bigoplus_{i<t}(\pm A_i)$ is invariant under negation, so negating the first identity gives the second. Since $[d] \cap (-[d]) = \varnothing$, (5) follows. Conversely, suppose (5) holds. For $(a_i) \in \prod_i A_i$ and $\varepsilon_i \in \{-1, 1\}$,

$$\sum_{i=1}^{t} \varepsilon_i a_i \equiv \sum_{i=1}^{t} a_i \pmod 2.$$

Thus all 2^t sign choices for a fixed tuple (a_i) have the same parity. Since $t \ge 2$, $d = 2^{t-1}\prod_i n_i$ is even. Counting the odd integers in (5) gives

$$\begin{aligned} d &= \#\{z \in \pm[d] : z \text{ is odd}\} \\ &= 2^t \#\{(a_i) \in \textstyle\prod_i A_i : \sum_i a_i \text{ is odd}\}. \end{aligned}$$

Hence $2^t \mid d$, so $\prod_i n_i$ is even and at least one n_i is even. □

C CKKS Details for SWIFT

This appendix proves the operation counts stated in Section 4. Unless stated otherwise, let $n = 2^m \geq 2$ and $d = (2n)^t/2$. The factor sets are

$$\begin{aligned} A_i &= \frac{n}{2}(2n)^{i-1}\{1, 3, \ldots, 2n-1\} \quad (1 \leq i < t), \\ A_t &= \left\{ \frac{d-n}{2} + 1, \ldots, \frac{d+n}{2} \right\}. \end{aligned} \tag{11}$$

Write $v_i := v_{A_i}$; using the induced coordinate order, $\overline{v_i} = v_{-A_i}$.

One CKKS ciphertext has $N/2$ complex slots. Assume $2n \mid N$ and $2tn \leq N$, and set $h := N/(2n)$ and $\beta := \lfloor N/(2tn) \rfloor$. The coordinates of an n-entry vector are placed h slots apart. For batch j, the k-th coordinate of factor i is therefore at slot $j + (i-1)\beta + kh$.

C.1 Detailed Slot Expansion

For $1 \leq i \leq t$, define

$$(\ell_{i,0}, \Delta\ell_i) = \begin{cases} \left(\frac{n}{2}(2n)^{i-1}, n(2n)^{i-1}\right), & 1 \leq i < t, \\ \left(\frac{d-n}{2} + 1, 1\right), & i = t. \end{cases} \tag{12}$$

The k-th exponent of A_i is $\ell_{i,k} = \ell_{i,0} + k\,\Delta\ell_i$. Algorithm 1 uses this formula in its packing step before ExpBTS.

Proposition 7 (Correctness and cost of slot expansion). *Algorithm 1 generates $v_1, \ldots, v_t$ for β inputs. Outside ExpBTS, the algorithm has multiplicative depth zero and uses $2(t-1) + 2\log_2 n$ rotations and at most $t + \log_2 n - 1$ nontrivial integer plaintext multiplications.*

Proof. Let $[\mathsf{ct}]_s$ denote the value in slot s with zero arithmetic error. For $1 \leq i \leq t$ and $0 \leq j < \beta$, put $s_{i,j,k} := j + (i-1)\beta + kh$. Since $t\beta \leq h$, the input blocks do not overlap. The initialization gives

$$[\mathsf{ct}_{\mathrm{base}}]_{s_{i,j,0}} = \ell_{i,0}x_j, \qquad [\mathsf{ct}_{\mathrm{step}}]_{s_{i,j,0}} = \Delta\ell_i x_j.$$

We show by induction that, after p expansion iterations,

$$\begin{aligned} [\mathsf{ct}_{\mathrm{base}}]_{s_{i,j,k}} &= (\ell_{i,0} + k\Delta\ell_i)x_j, \\ [\mathsf{ct}_{\mathrm{step}}]_{s_{i,j,k}} &= \Delta\ell_i x_j, \qquad 0 \leq k < 2^p. \end{aligned}$$

The initialization proves the case $p = 0$. At iteration p, rotation by $2^p h$ places

$$(\ell_{i,0} + k\,\Delta\ell_i)x_j + 2^p\Delta\ell_i x_j = (\ell_{i,0} + (k + 2^p)\Delta\ell_i)x_j$$

at slot $s_{i,j,k+2^p}$; the corresponding entry of $\mathsf{ct}_{\mathrm{step}}$ remains $\Delta\ell_i x_j$. The entries with $k < 2^p$ are unchanged, proving the induction step. At $p = m$, $2^m = n$, so ExpBTS and factor alignment give

$$\ell_{i,k}x_j \longmapsto e^{2\pi\mathrm{i}\ell_{i,k}x_j} = \alpha_j^{\ell_{i,k}}, \qquad [\mathsf{ct}_i]_{j+kh} = \alpha_j^{\ell_{i,k}}.$$

The costs are

$$\#\mathsf{rot} = \underbrace{t-1}_{\text{input placement}} + \underbrace{2m}_{\text{expansion}} + \underbrace{t-1}_{\text{factor alignment}},$$
$$\#\text{integer scalings} \le t + (m-1), \qquad D = 0.$$

The initialization uses at most t integer scalings, with $2\mathsf{ct}_{\text{reg}}$ formed by addition. The expansion uses the $m-1$ scalings 2^p for $1 \le p < m$. All these operations have multiplicative depth zero. □

The $t = 1$ alignment. When $t = 1$, $d = n$ and $A_1 = \{1, \dots, n\}$. Initialize $\mathsf{ct}_{\text{base}} = \mathsf{ct}_{\text{step}} = \mathsf{ct}_x$. In the first expansion, share $\rho = \mathsf{rot}_h(\mathsf{ct}_x)$ and set

$$\mathsf{ct}_{\text{base}} \leftarrow \mathsf{ct}_x + 2\rho, \qquad \mathsf{ct}_{\text{step}} \leftarrow \mathsf{ct}_x + \rho.$$

The remaining $m-1$ stages use the rotate-and-add recurrence from Algorithm 1. This places $x_j, 2x_j, \dots, nx_j$ using $2\log_2 n - 1$ rotations at depth zero. One ExpBTS call then produces v_1, with no factor realignment.

C.2 Tensor Product of Ciphertext Lists

Suppose $\mathsf{CT}_w = (\mathsf{ct}_{w,1}, \dots, \mathsf{ct}_{w,a})$ encrypts the n-entry blocks of $w \in \mathbb{C}^{an}$, and $\mathsf{CT}_u = (\mathsf{ct}_{u,1}, \dots, \mathsf{ct}_{u,b})$ encrypts the n-entry blocks of $u \in \mathbb{C}^{bn}$. In every block, consecutive coordinates are $h = N/(2n)$ slots apart. For positive integers a, b, define

$$\tau_n(a,b) := \min_{\substack{n_1 n_2 = n \\ n_1, n_2 \in \mathbb{Z}_{\ge 1}}} \big\{ a(n_2 - 1) + b(n_1 - 1) \big\}.$$

Algorithm 2 chooses n_1, n_2 attaining the minimum. The rotations also permute the coordinates within each output block. The permutation is fixed by n_1, n_2, so the rows and columns of C are permuted before evaluation to match the output order.

Proposition 8 (Tensor cost). *Algorithm 2 outputs abn ciphertexts. It has multiplicative depth one and uses $\tau_n(a,b)$ rotations and abn ciphertext multiplications.*

Proof. Fix a pair (n_1, n_2) attaining the minimum in $\tau_n(a,b)$. Let $w_i, u_j \in \mathbb{C}^n$ be the vector blocks encrypted by $\mathsf{ct}_{w,i}, \mathsf{ct}_{u,j}$. For each i, j, Algorithm 2 computes

$$\mathsf{rot}_{-qn_1 h}(\mathsf{ct}_{w,i}) \cdot \mathsf{rot}_{ph}(\mathsf{ct}_{u,j}), \qquad 0 \le q < n_2, \quad 0 \le p < n_1.$$

The corresponding vector product satisfies

$$\mathsf{rot}_{-qn_1}(w_i) \odot \mathsf{rot}_p(u_j) = \mathsf{rot}_{-qn_1}\big(w_i \odot \mathsf{rot}_{qn_1+p}(u_j)\big).$$

The map $(q,p) \mapsto qn_1 + p : \{0, \dots, n_2 - 1\} \times \{0, \dots, n_1 - 1\} \to \{0, \dots, n-1\}$ is bijective. Hence the products contain every coordinate of $w_i \otimes u_j$ once, up to

Algorithm 3 TensorTree: tensor reconstruction

Require: An ordered list $\mathcal{L} = (\mathcal{L}_1, \ldots, \mathcal{L}_r)$ of factor-block lists.
Ensure: An ordered ciphertext-block list representing $\bigotimes_{i=1}^{r} \mathrm{concat}(\mathcal{L}_i)$.
1: **if** $r = 1$ **then**
2: **return** $\mathcal{L}_1$
3: **end if**
4: $r_+ \leftarrow \lceil r/2 \rceil$, $r_- \leftarrow \lfloor r/2 \rfloor$
5: $\mathsf{CT}_w \leftarrow \mathsf{TensorTree}((\mathcal{L}_i)_{i=1}^{r_+})$
6: $\mathsf{CT}_u \leftarrow \mathsf{TensorTree}((\mathcal{L}_i)_{i=r_++1}^{r})$
7: **return** $\mathsf{Tensor}(\mathsf{CT}_w, \mathsf{CT}_u)$

Cost. For r factors, the depth is $\lceil \log_2 r \rceil$. The recurrences below give the rotation and multiplication counts. For v_G, place u_t first in $\mathcal{L}$, so u_t remains in $(\mathcal{L}_i)_{i=1}^{r_+}$ at each recursive call containing u_t.

the permutations rot_{-qn_1}. The counts are

$$\begin{aligned} \#\text{output blocks} &= abn^2/n = abn, \\ \#\mathsf{rot} &= a(n_2-1) + b(n_1-1) = \tau_n(a,b), \\ \#\mathsf{ctmult} &= abn_1n_2 = abn, \qquad D = 1. \end{aligned}$$

□

C.3 Reconstruction of v_B and v_G

Assume $t \geq 2$. The $t = 1$ evaluation is given in Section C.5. Let $b, g \geq 1$ be integers with $b + g = t$. Define

$$u_i = (v_i \| \overline{v_i}) \quad (i < t), \qquad u_t = v_t.$$

Then

$$v_B \cong \bigotimes_{i=1}^{b} u_i, \qquad v_G \cong u_t \otimes \bigotimes_{b<i<t} u_i. \tag{13}$$

Each u_i for $i < t$ occupies two ciphertext blocks, and u_t occupies one.

For $r \geq 1$ factors of the form u_i with $i < t$, let $L_B(r) := (2n)^r/n = 2(2n)^{r-1}$ be the number of ciphertext blocks. Let $M_B(r)$ and $K_B(r)$ denote the ciphertext multiplications and rotations of Algorithm 3. Put $r_+ := \lceil r/2 \rceil$ and $r_- := \lfloor r/2 \rfloor$. For $r \geq 2$,

$$\begin{aligned} M_B(1) &= K_B(1) = 0, \\ M_B(r) &= M_B(r_+) + M_B(r_-) + L_B(r), \\ K_B(r) &= K_B(r_+) + K_B(r_-) + \tau_n\big(L_B(r_+), L_B(r_-)\big). \end{aligned} \tag{14}$$

The last tensor product uses $nL_B(r_+)L_B(r_-) = L_B(r)$ ciphertext multiplications.

For $r \geq 1$ factors including u_t, let $L_G(r) := (2n)^{r-1}$, and define $M_G(r), K_G(r)$ in the same way. Put u_t among the r_+ factors defining the G product; the r_- factors defining the B product exclude u_t. For $r \geq 2$,

$$\begin{aligned} M_G(1) &= K_G(1) = 0, \\ M_G(r) &= M_G(r_+) + M_B(r_-) + L_G(r), \\ K_G(r) &= K_G(r_+) + K_B(r_-) + \tau_n\big(L_G(r_+), L_B(r_-)\big). \end{aligned} \tag{15}$$

Here $nL_G(r_+)L_B(r_-) = L_G(r)$.

Proposition 9 (Reconstruction of v_B and v_G). *The recursions construct the vectors in Equation* (13) *using*

$$R := \frac{|B|}{n} = 2(2n)^{b-1}, \qquad S := \frac{|G|}{n} = (2n)^{g-1} \tag{16}$$

ciphertext blocks. They use $M_B(b) + M_G(g)$ ciphertext multiplications, $K_B(b) + K_G(g)$ rotations, and $t-1$ conjugations. Their depth is $\max\{\lceil \log_2 b \rceil, \lceil \log_2 g \rceil\}$.

Proof. Since $\dim u_i = 2n$ for $i < t$ and $\dim u_t = n$,

$$R = \frac{(2n)^b}{n}, \qquad S = \frac{n(2n)^{g-1}}{n}, \qquad \#\mathsf{conj} = t - 1.$$

Proposition 8 gives the recurrences (14) and (15). For $r_+ = \lceil r/2 \rceil$, $r_- = \lfloor r/2 \rfloor$, the depth satisfies

$$D(1) = 0, \qquad D(r) = 1 + \max\{D(r_+), D(r_-)\} = \lceil \log_2 r \rceil.$$

□

Proof of Proposition 2. Fix b, g with $b + g \geq 3$, and let $n \to \infty$ through powers of two. Put $q := \max\{b, g\} \geq 2$, $r_+ := \lceil r/2 \rceil$, and $r_- := \lfloor r/2 \rfloor$. For a product of $r \geq 2$ factors, the final tensor multiplication has input block counts

$$(a_r, b_r) = (L_B(r_+), L_B(r_-)) \quad \text{or} \quad (a_r, b_r) = (L_G(r_+), L_B(r_-)).$$

For $r \geq 2$, minimizing $a_r(n/n_1 - 1) + b_r(n_1 - 1)$ gives

$$\begin{aligned} r = 2k: \quad & a_r, b_r = \Theta((2n)^{k-1}), \quad n_1, n_2 = \Theta(\sqrt{n}), \\ & \tau_n(a_r, b_r) = \Theta(\sqrt{n}(2n)^{k-1}), \\ r = 2k+1: \quad & a_r = \Theta((2n)^k), \quad b_r = \Theta((2n)^{k-1}), \\ & n_1 = \Theta(n), \quad n_2 = \Theta(1), \quad \tau_n(a_r, b_r) = \Theta((2n)^k). \end{aligned}$$

Substituting these bounds into the rotation recurrences gives, for $r \geq 2$,

$$K_B(r), K_G(r) = \Theta\Big(\sqrt{n}(2n)^{r/2-1}\Big).$$

Since $K_B(1) = K_G(1) = 0$ and $q \geq 2$,

$$K_B(b) + K_G(g) = o((2n)^{q-1}).$$

For ciphertext multiplications, $r_+, r_- < r$ implies

$$\begin{aligned} M_B(r) &= L_B(r) + M_B(r_+) + M_B(r_-) = L_B(r) + o((2n)^{r-1}), \\ M_G(r) &= L_G(r) + M_G(r_+) + M_B(r_-) = L_G(r) + o((2n)^{r-1}) \qquad (r \geq 2). \end{aligned}$$

For $r = 1$, $M_B(1) = M_G(1) = 0$, whereas $L_B(1) = 2$ and $L_G(1) = 1$. The differences are $O(1) = o((2n)^{q-1})$. Counting one relinearization per ciphertext multiplication gives

$$\begin{aligned} \#\mathsf{ks} &= M_B(b) + M_G(g) + K_B(b) + K_G(g) + (t-1) \\ &= 2(2n)^{b-1} + (2n)^{g-1} + o((2n)^{q-1}). \end{aligned}$$

□

C.4 PCMV and the Encrypted Inner Product

This subsection gives the rotations and plaintext multiplications used by PCMV, followed by the ciphertext multiplications and rotations used by CtInnerProd. Recall that

$$z_{[d]}(\alpha) = \langle C v_G, v_B \rangle, \qquad f(x) = c_0 + z_{[d]}(\alpha) + \overline{z_{[d]}(\alpha)}, \tag{17}$$

where $C \in \mathbb{C}^{nR \times nS}$. Thus C is an $R \times S$ block matrix with $n \times n$ plaintext blocks. For a block M, let $\mathrm{diag}_r(M) := (M_{u,u-r \bmod n})_{u=0}^{n-1}$ be its r-th cyclic diagonal. If $n = n_{\mathrm{BS}} n_{\mathrm{GS}}$ and $r = n_{\mathrm{BS}} k_2 + k_1$, precompute

$$\widetilde{\mathrm{diag}}_{k_1,k_2}(M) := \mathsf{rot}_{-n_{\mathrm{BS}} k_2 h}\big(\mathrm{diag}_{n_{\mathrm{BS}} k_2 + k_1}(M)\big).$$

Proposition 10 (Cost of PCMV). *Algorithm 4 has multiplicative depth one and costs*

$$\begin{aligned} S(n_{\mathrm{BS}} - 1) + R(n_{\mathrm{GS}} - 1) \quad & \textit{rotations}, \\ nRS = (2n)^{t-1} \quad & \textit{plaintext multiplications}. \end{aligned} \tag{18}$$

Over real values $1 \leq n_{\mathrm{BS}} \leq n$, the rotation count is minimized at $n_{\mathrm{BS}} = \min\{n, \max\{1, \sqrt{nR/S}\}\}$. The algorithm selects the positive divisor of n that minimizes the same count.

Proof. For $M \in \mathbb{C}^{n \times n}$ and $v \in \mathbb{C}^n$, the cyclic-diagonal decomposition gives

$$Mv = \sum_{k_2=0}^{n_{\mathrm{GS}}-1} \mathsf{rot}_{n_{\mathrm{BS}} k_2} \left(\sum_{k_1=0}^{n_{\mathrm{BS}}-1} \mathsf{rot}_{-n_{\mathrm{BS}} k_2}(\mathrm{diag}_{n_{\mathrm{BS}} k_2 + k_1}(M)) \odot \mathsf{rot}_{k_1}(v) \right).$$

Algorithm 4 $\mathsf{PCMV}_{n_{\mathrm{BS}},n_{\mathrm{GS}}}$

Require: Plaintext blocks $C_{r,j} \in \mathbb{C}^{n\times n}$ for $1 \le r \le R$, $1 \le j \le S$; ciphertext blocks $\mathsf{CT}_G = (\mathsf{ct}_{G,1}, \ldots, \mathsf{ct}_{G,S})$; and $n = n_{\mathrm{BS}} n_{\mathrm{GS}}$.
Ensure: A ciphertext-block list $\mathsf{CT}_u = (\mathsf{ct}_{u,1}, \ldots, \mathsf{ct}_{u,R})$ encrypting Cv_G.

for $j = 1, \ldots, S$ **do**
  $\rho_{j,0} \leftarrow \mathsf{ct}_{G,j}$
  for $k_1 = 1, \ldots, n_{\mathrm{BS}} - 1$ **do**
    $\rho_{j,k_1} \leftarrow \mathsf{rot}_{k_1 h}(\mathsf{ct}_{G,j})$
  end for
end for
for $r = 1, \ldots, R$ **do**
  for $k_2 = 0, \ldots, n_{\mathrm{GS}} - 1$ **do**
    $\sigma_{r,k_2} \leftarrow 0$
    for $j = 1, \ldots, S$ **do**
      for $k_1 = 0, \ldots, n_{\mathrm{BS}} - 1$ **do**
        $\sigma_{r,k_2} \leftarrow \sigma_{r,k_2} + \widetilde{\mathrm{diag}}_{k_1,k_2}(C_{r,j}) \cdot \rho_{j,k_1}$
      end for
    end for
  end for
  $\mathsf{ct}_{u,r} \leftarrow \sigma_{r,0}$
  for $k_2 = 1, \ldots, n_{\mathrm{GS}} - 1$ **do**
    $\mathsf{ct}_{u,r} \leftarrow \mathsf{ct}_{u,r} + \mathsf{rot}_{n_{\mathrm{BS}} k_2 h}(\sigma_{r,k_2})$
  end for
end for
return CT_u

Apply the identity to $M = C_{r,j}$ and sum over j. The output block is $\sum_j C_{r,j} g_j$. Input rotations are shared over the R output blocks, giving

$$\#\mathsf{rot} = \underbrace{S(n_{\mathrm{BS}} - 1)}_{\text{input blocks}} + \underbrace{R(n_{\mathrm{GS}} - 1)}_{\text{output blocks}},$$
$$\#\mathsf{ptmult} = RSn_{\mathrm{BS}} n_{\mathrm{GS}} = nRS, \qquad D = 1.$$

For a real variable $\xi \in [1, n]$, define $F(\xi) := S(\xi - 1) + R(n/\xi - 1)$. Then

$$F'(\xi) = S - Rn/\xi^2, \qquad F''(\xi) = 2Rn/\xi^3 > 0,$$
$$\operatorname*{argmin}_{1\le\xi\le n} F(\xi) = \min\{n, \max\{1, \sqrt{nR/S}\}\}.$$

The algorithm chooses $n_{\mathrm{BS}} \mid n$ minimizing $F(n_{\mathrm{BS}})$ and sets $n_{\mathrm{GS}} = n/n_{\mathrm{BS}}$. □

Both $\mathsf{PCMV}(C, \mathsf{CT}_G)$ and CT_B are ciphertext lists. Algorithm 5 computes their inner product.

The total depth is

$$\max\left\{\lceil \log_2 b \rceil + 1,\ \lceil \log_2 g \rceil + 2\right\}. \tag{19}$$

Algorithm 5 CtInnerProd: inner product of encrypted block lists

Require: Modulus- and scale-aligned lists $\mathsf{CT}_u = (\mathsf{ct}_{u,1}, \dots, \mathsf{ct}_{u,R})$ and $\mathsf{CT}_B = (\mathsf{ct}_{B,1}, \dots, \mathsf{ct}_{B,R})$.
Ensure: A ciphertext whose slot j encrypts $\sum_{r=1}^{R} \langle u_r, b_r \rangle$ for every $0 \le j < \beta$.
1: $\mathsf{ct}_w \leftarrow \sum_{r=1}^{R} \mathsf{ct}_{u,r} \cdot \mathsf{ct}_{B,r}$
2: **for** $p = 0, \dots, m-1$ **do**
3: $\quad \mathsf{ct}_w \leftarrow \mathsf{ct}_w + \mathsf{rot}_{2^p h}(\mathsf{ct}_w)$
4: **end for**
5: **return** ct_w

Cost. The algorithm has multiplicative depth one and uses R ciphertext multiplications and $\log_2 n$ rotations.

The computation of CtInnerProd adds one level after CT_B. The computation of PCMV, followed by CtInnerProd, adds two levels after CT_G.

After the two operations, ct_z encrypts $z_{[d]}(\alpha)$, and we return $\mathsf{ct}_f := \mathsf{ct}_z + \overline{\mathsf{ct}_z} + c_0$. This step uses one conjugation and no multiplicative level. Together with the $t-1$ conjugations used to form $\overline{v_i}$ for $i < t$, the evaluation uses t conjugations.

Proof of Proposition 3. Put $r := n_{\mathrm{BS}}$, $R := 2(2n)^{b-1}$, and $S := (2n)^{g-1}$. Adding the rotations, one relinearization per ciphertext multiplication, and the final conjugation gives

$$\begin{aligned} K(r) &:= S(r-1) + R(n/r - 1) + R + \log_2 n + 1 \\ &= S(r-1) + Rn/r + \log_2 n + 1, \\ \#\mathsf{ptmult} &= nRS = (2n)^{t-1}. \end{aligned}$$

Since $K'(r) = S - Rn/r^2$ and $K''(r) > 0$,

$$\begin{gathered} r_* = \min\{n, \max\{1, (2n)^{(b-g+1)/2}\}\}, \\ b < g \implies r_* = 1, \qquad b > g \implies r_* = n. \end{gathered}$$

For $b = g$, put $r = 2^k$ and $F(k) = 2^k + 2^{m+1-k}$. Then

$$F(k+1) - F(k) = 2^k - 2^{m-k}, \qquad \min_{0 \le k \le m} F(k) = F(\lceil m/2 \rceil).$$

Substituting the minimizing choices gives

$$\begin{aligned} K(1) &= (2n)^b + \log_2 n + 1, \\ K(2^{\lceil m/2 \rceil}) &= \left(2^{\lceil m/2 \rceil} + 2^{\lfloor m/2 \rfloor + 1} - 1\right)(2n)^{b-1} + \log_2 n + 1, \\ K(n) &= 2(2n)^{b-1} + \tfrac{1}{2}(2n)^g - (2n)^{g-1} + \log_2 n + 1. \end{aligned}$$

For fixed b, g and $n \to \infty$,

$$\begin{aligned} \log_2 n + 1 &= o((2n)^b), \\ (2n)^{b-1} + \log_2 n + 1 &= o((2n)^{b-1/2}), \\ (2n)^{g-1} + \log_2 n + 1 &= o((2n)^{b-1}) \quad (b > g). \end{aligned}$$

Algorithm 6 SWIFTEval: tensor evaluation

Require: Input ciphertext ct_x and coefficient c_0; additionally, the packed coefficient plaintext p_c if $t = 1$, or the coefficient matrix C and integers $b, g \geq 1$ with $b+g = t$ if $t \geq 2$.

Ensure: A ciphertext ct_f whose slot j decrypts to an approximation of $f(x_j)$ for every $0 \leq j < \beta$.

$(\mathsf{ct}_1, \ldots, \mathsf{ct}_t) \leftarrow \mathsf{FactorGen}(\mathsf{ct}_x)$
if $t = 1$ **then**
  $\mathsf{ct}_w \leftarrow p_c \cdot \mathsf{ct}_1$
  for $p = 0, \ldots, \log_2 d - 1$ **do**
    $\mathsf{ct}_w \leftarrow \mathsf{ct}_w + \mathsf{rot}_{2^p h}(\mathsf{ct}_w)$
  end for
  $\mathsf{ct}_z \leftarrow \mathsf{ct}_w$
else
  $\mathcal{L}_i \leftarrow (\mathsf{ct}_i, \overline{\mathsf{ct}_i}) \quad (i = 1, \ldots, t-1)$
  $\mathcal{L}_t \leftarrow (\mathsf{ct}_t)$
  $\mathcal{G} \leftarrow (\mathcal{L}_t, (\mathcal{L}_i)_{b<i<t})$
  $(\mathsf{CT}_B, \mathsf{CT}_G) \leftarrow \big(\mathsf{TensorTree}((\mathcal{L}_i)_{i=1}^{b}), \mathsf{TensorTree}(\mathcal{G})\big)$
  $\mathsf{CT}_u \leftarrow \mathsf{PCMV}(C, \mathsf{CT}_G)$
  $\mathsf{ct}_z \leftarrow \mathsf{CtInnerProd}(\mathsf{CT}_u, \mathsf{CT}_B)$
end if
$\mathsf{ct}_f \leftarrow \mathsf{ct}_z + \overline{\mathsf{ct}_z} + c_0$
return ct_f

These estimates give the three stated asymptotic expansions. □

C.5 SWIFT Evaluation

Algorithm 6 uses FactorGen. For $t = 1$, FactorGen uses the construction in the $t = 1$ alignment paragraph above.

For $t = 1$, recall that $d = n$ and $A_1 = [d]$. Let p_c be the plaintext whose slot $j+kh$ contains c_{k+1}, for $0 \leq j < \beta$ and $0 \leq k < d$, and whose other slots contain zero. Multiplication by p_c, followed by $\log_2 d$ rotate-and-add steps, computes $\sum_{\ell=1}^{d} c_\ell \alpha^\ell$. For $t \geq 2$, the algorithm uses CT_B, CT_G, PCMV, and CtInnerProd.

Proposition 11 (Correctness of SWIFTEval). *If every CKKS operation returns the sum, product, conjugation, or rotation specified in Algorithm 6, then slot j of its output encrypts $f(x_j)$ for every $0 \leq j < \beta$.*

Proof. Let $[\mathsf{ct}]_s$ denote the value in slot s with zero arithmetic error. Fix $0 \leq j < \beta$. Algorithm 1 gives

$$[\mathsf{ct}_i]_{j+kh} = \alpha_j^{\ell_{i,k}} \qquad (1 \leq i \leq t,\ 0 \leq k < n).$$

For $t = 1$, $\ell_{1,k} = k + 1$, so

$$[p_c \mathsf{ct}_1]_{j+kh} = c_{k+1}\alpha_j^{k+1}, \qquad [\mathsf{ct}_z]_j = \sum_{k=0}^{d-1} c_{k+1}\alpha_j^{k+1} = z_{[d]}(\alpha_j).$$

For $t \geq 2$, conjugation and tensor multiplication produce v_B, v_G. The outputs of PCMV and CtInnerProd satisfy

$$[\mathsf{ct}_z]_j = v_B^\top C v_G = z_{[d]}(\alpha_j).$$

In both cases, the final conjugation and additions give

$$[\mathsf{ct}_f]_j = c_0 + [\mathsf{ct}_z]_j + \overline{[\mathsf{ct}_z]_j} = c_0 + z_{[d]}(\alpha_j) + \overline{z_{[d]}(\alpha_j)} = f(x_j).$$

□

C.6 Total Cost

The counts below are for operations outside ExpBTS. Here PtMult counts plaintext multiplications in the coefficient evaluation; integer scalings in factor generation are counted separately in Subsection C.1.

For $t = 1$, we have $n = d$ and batch size $\lfloor N/(2d) \rfloor$. Combining the alignment for $t = 1$ with the coefficient evaluation in Algorithm 6 gives

$$\mathsf{Rot} = 3\log_2 d - 1, \qquad \mathsf{PtMult} = 1, \qquad \mathsf{CtMult} = 0, \qquad \mathsf{Conj} = 1, \tag{20}$$

with one ExpBTS call and post-bootstrap multiplicative depth one.

For $t \geq 2$, let $b + g = t$, with $b, g \geq 1$, and let $n = n_{\mathrm{BS}} n_{\mathrm{GS}}$. Combining the preceding propositions, the construction uses

$$\begin{aligned} \mathsf{Rot} &= 2(t-1) + 2\log_2 n + K_B(b) + K_G(g) \\ &\quad + S(n_{\mathrm{BS}} - 1) + R(n_{\mathrm{GS}} - 1) + \log_2 n, \\ \mathsf{CtMult} &= M_B(b) + M_G(g) + R, \\ \mathsf{PtMult} &= nRS = (2n)^{t-1}, \qquad \mathsf{Conj} = t, \qquad \mathsf{ExpBTSCalls} = 1. \end{aligned} \tag{21}$$

Each call to Algorithm 2 selects the divisors in τ_n for the two input lists of that call. The divisors $n_{\mathrm{BS}}, n_{\mathrm{GS}}$ are selected separately for PCMV.

The batch size is $\lfloor N/(2tn) \rfloor$, and the post-ExpBTS depth is given in (19). For fixed t, we have

$$2n = (2d)^{1/t}, \qquad nRS = (2n)^{t-1} = (2d)^{1-1/t}.$$

Put $r_0 = \sqrt{nR/S}$ and $F(r) = S(r-1) + R(n/r - 1)$. Choosing $r \mid n$ gives

$$\begin{aligned} 1 \leq r_0 \leq n: &\quad r \in [r_0/2, 2r_0], &\quad F(r) &= O(\sqrt{nRS}), \\ r_0 > n: &\quad r = n, &\quad F(n) &= S(n-1) \leq \sqrt{nRS}, \\ r_0 < 1: &\quad r = 1, &\quad F(1) &= R(n-1) \leq \sqrt{nRS}. \end{aligned}$$

In the first line, such a divisor exists because n is a power of two. Hence

$$S(n_{\mathrm{BS}} - 1) + R(n_{\mathrm{GS}} - 1) = O\Big((2d)^{1/2 - 1/(2t)}\Big).$$

This bound counts the PCMV rotations. Equations (14) and (15) count the tensor-reconstruction operations.

Proof of Theorem 1. Let $b, g \geq 1$ with $b + g = t$. By (19),

$$D(b,g) \leq D \iff b \leq 2^{D-1}, \quad g \leq 2^{D-2},$$
$$\min_{b+g=t} D(b,g) = \min\{D \geq 2 : t \leq 3 \cdot 2^{D-2}\} = D_t.$$

Put $G_{\max} := 2^{D_t-2}$, $g_* := \min\{\lceil t/2 \rceil, G_{\max}\}$, and $b_* := t - g_*$. The pair (b_*, g_*) satisfies the depth constraints because

$$g_* \leq G_{\max},$$
$$g_* = G_{\max} \implies b_* = t - G_{\max} \leq 2G_{\max},$$
$$g_* = \lceil t/2 \rceil \implies b_* = \lfloor t/2 \rfloor \leq G_{\max}.$$

For fixed b, g, first sum the reconstruction and bilinear-contraction costs in Propositions 2 and 3. Their key-switching count is $\Theta((2n)^{k(b,g)})$, where

$$b < g: \quad k(b,g) = \max\{g-1, b\},$$
$$b = g: \quad k(b,g) = b - \tfrac{1}{2},$$
$$b > g: \quad k(b,g) = b - 1.$$

For $t = 2$, tensor reconstruction uses one conjugation, and the same formula follows from Proposition 3. In all three cases,

$$k(b,g) \geq \frac{t-1}{2}, \qquad 2(t-1) + 2\log_2 n = o((2n)^{k(b,g)}).$$

Hence the factor-generation rotations do not change the exponent $k(b,g)$ in the total key-switching count. If $G_{\max} \geq \lceil t/2 \rceil$, then $k(b_*, g_*) = (t-1)/2$. If $G_{\max} < \lceil t/2 \rceil$, every pair of depth D_t satisfies

$$g \leq G_{\max}, \quad b = t - g \geq g, \qquad k(t-g, g) \geq k(t - G_{\max}, G_{\max}).$$

The inequality follows from the two formulas for $b \geq g$ above. Using $2n = (2d)^{1/t}$, we obtain

$$k(b_*, g_*) = \min_{\substack{b+g=t \\ D(b,g)=D_t}} k(b,g) = \max\{b_* - 1, (t-1)/2\},$$
$$\#\mathsf{ks} = \Theta\Big((2d)^{\max\{b_*-1,(t-1)/2\}/t}\Big),$$
$$\#\mathsf{ptmult} = nRS = (2d)^{1-1/t}, \qquad \#\mathsf{ExpBTS} = 1.$$

□

D Proofs for Error Analysis

We use g_i, e_i and the ℓ_∞ norm from Subsection 4.5. Assume

$$x \in \mathbb{R}, \qquad t \geq 1 \text{ fixed}, \qquad \|c\|_1 = \Omega(1), \qquad \epsilon_{\mathsf{BTS}} + d^{1+1/t}\epsilon_m = o(1).$$

The ϵ_{BTS} bound holds at every input supplied to ExpBTS. We propagate the packing error through the exponential below. Every other arithmetic operation introduces slotwise output error at most ϵ_m. The asymptotic constants may depend on t, but are independent of x, d, $\|c\|_1$, ϵ_{BTS}, and ϵ_m.

D.1 Local Error Bounds

Proof of Proposition 4. We bound e_1, e_2, e_3, e_4 separately.

Factor generation. In Algorithm 1, let E_{reg} bound the error in $\mathsf{ct}_{\mathrm{reg}}$. Let B_p, S_p bound the errors in $\mathsf{ct}_{\mathrm{base}}, \mathsf{ct}_{\mathrm{step}}$ after p expansion iterations. All errors are measured in ℓ_∞ relative to the same computations without arithmetic error. For $t \geq 2$, the sum of the coefficients used to form $\mathsf{ct}_{\mathrm{reg}}$ is

$$a := \sum_{i=1}^{t-1} \frac{n}{2}(2n)^{i-1} = \frac{d-n}{2(2n-1)} \geq 2t-3.$$

The initialization therefore satisfies

$$E_{\mathrm{reg}} \leq (a+2t-3)\epsilon_m \leq 2a\epsilon_m, \qquad B_0 \leq d\epsilon_m, \qquad S_0 \leq (2d/n)\epsilon_m.$$

For $0 \leq p < m = \log_2 n$, the expansion step gives

$$S_{p+1} \leq 2S_p + 2\epsilon_m, \qquad S_p \leq 2^p(2d/n+2)\epsilon_m,$$
$$B_{p+1} \leq 2B_p + 2^p S_p + 4\epsilon_m.$$

Therefore, using $d \geq 2n^2$ for $t \geq 2$,

$$\begin{aligned} B_m &\leq nB_0 + \sum_{p=0}^{m-1} 2^{m-1-p}(2^p S_p + 4\epsilon_m) \\ &\leq \big[(2n-1)d + (n-1)(n+4)\big]\epsilon_m \leq 2nd\epsilon_m. \end{aligned}$$

For $t = 1$, $d = n$ and $B_0 = S_0 = 0$, so

$$B_m \leq (n-1)(n+4)\epsilon_m \leq 2n^2\epsilon_m.$$

The shared rotation for $t = 1$ in Appendix C.1 satisfies the same bounds. For a packed value $y \in \mathbb{R}$ and error $\zeta \in \mathbb{C}$,

$$\left| e^{2\pi \mathrm{i}(y+\zeta)} - e^{2\pi \mathrm{i} y} \right| \leq e^{2\pi|\zeta|} - 1, \qquad |\zeta| \leq B_m \leq 2nd\epsilon_m.$$

The ExpBTS call and the factor-alignment rotation add at most ϵ_{BTS} and ϵ_m, respectively. Hence

$$\begin{aligned} \|e_1\|_\infty &\leq \epsilon_{\mathsf{BTS}} + e^{4\pi nd\epsilon_m} - 1 + \epsilon_m \\ &= O(\epsilon_{\mathsf{BTS}} + d^{1+1/t}\epsilon_m) = o(1), \end{aligned}$$

since $n = \Theta(d^{1/t})$ and $nd\epsilon_m = o(1)$.

Tensor products. Put $A := 1 + \|e_1\|_\infty$, so every entry of $\widetilde{x}_1$ has magnitude at most A. For a product of r factors from $\widetilde{x}_1$, let E_r bound the arithmetic error relative to the exact tensor product of those factors. Let H_r bound the computed output magnitude. We prove by induction that

$$E_r \le A^r\big((1+\epsilon_m)^{4r-3} - 1\big), \qquad H_r \le A^r(1+\epsilon_m)^{4r-3}.$$

For $r = 1$, at most one conjugation is applied to a factor:

$$E_1 \le \epsilon_m \le A\epsilon_m, \qquad H_1 \le A + \epsilon_m \le A(1+\epsilon_m).$$

Consider a tensor multiplication of products with r and s factors. The two rotations and one multiplication introduce errors ρ, σ, μ with $|\rho|, |\sigma|, |\mu| \le \epsilon_m$. For each output coordinate, write

$$\widehat{u} = u + \Delta u, \quad \widehat{v} = v + \Delta v, \quad |u| \le A^r, \quad |v| \le A^s, \quad |\Delta u| \le E_r, \quad |\Delta v| \le E_s.$$

Then

$$\begin{aligned} |(\widehat{u}+\rho)(\widehat{v}+\sigma) + \mu - uv| &\le A^r E_s + A^s E_r + E_r E_s \\ &\quad + \epsilon_m(H_r + H_s) + \epsilon_m^2 + \epsilon_m \\ &\le A^{r+s}\big((1+\epsilon_m)^{4(r+s)-3} - 1\big), \\ |(\widehat{u}+\rho)(\widehat{v}+\sigma) + \mu| &\le A^{r+s}(1+\epsilon_m)^{4(r+s)-3}. \end{aligned}$$

For the last two inequalities, substitute the bounds for E_r, E_s, H_r, H_s and use $(4r-3) + (4s-3) + 3 = 4(r+s) - 3$. Since $b, g \le t$,

$$\|e_2\|_\infty \le A^t\big((1+\epsilon_m)^{4t} - 1\big) = O(\epsilon_m).$$

The same induction bounds the exact tensor products by A^t and the computed tensor products by $A^t(1+\epsilon_m)^{4t}$. Thus every magnitude in the definition of δ is at most $A^t(1+\epsilon_m)^{4t} = 1 + o(1)$, and $\delta = o(1)$. For $t = 1$, Φ_2 is the identity and $e_2 = 0$.

Coefficient evaluation. Write $(\widetilde{v}_B, \widetilde{v}_G) := \widetilde{x}_2$. For output block r of Algorithm 4, define

$$C_r := \sum_{j=1}^{S} \sum_{k,l=1}^{n} |(C_{r,j})_{k,l}|, \qquad \sum_{r=1}^{R} C_r = \|c\|_1.$$

Let $E_{u,r}$ bound the ℓ_∞ error in the computed block r relative to $(C\widetilde{v}_G)_r$. The input-rotation errors are multiplied by entries of $C_{r,j}$; their total contribution is at most $C_r\epsilon_m$. The plaintext multiplications, additions, and output rotations give

$$E_{u,r} \le (C_r + 2nS + 2n_{\mathrm{GS}})\epsilon_m \le (C_r + 4nS)\epsilon_m,$$

where $n_{\mathrm{GS}} \le nS$. Let F_0 bound the error after the R ciphertext products are summed, before summing slots. Let F_p bound the error after p rotate-and-add

steps. These errors are measured relative to the exact bilinear evaluation at $(\widetilde{v}_B, \widetilde{v}_G)$. Since $\|\widetilde{v}_B\|_\infty \le 1+\delta$,

$$\begin{aligned} F_0 &\le (1+\delta)\sum_{r=1}^{R} E_{u,r} + 2R\epsilon_m \\ &\le \big[(1+\delta)(\|c\|_1 + 4nRS) + 2R\big]\epsilon_m, \\ F_{p+1} &\le 2F_p + 2\epsilon_m, \qquad F_m \le nF_0 + 2(n-1)\epsilon_m. \end{aligned}$$

Using $n^2RS = d$, $nR \le d$, and $\delta = o(1)$,

$$\begin{aligned} |e_3| &\le \big[n(1+\delta)\|c\|_1 + 4(1+\delta)d + 2nR + 2(n-1)\big]\epsilon_m \\ &= O\big((d^{1/t}\|c\|_1 + d)\epsilon_m\big). \end{aligned}$$

For $t = 1$, $n = d$ and the single plaintext multiplication gives $F_0 \le \epsilon_m$. The same slot-summation recurrence gives

$$|e_3| \le d\epsilon_m + 2(d-1)\epsilon_m = (3d-2)\epsilon_m.$$

Final output. The map $\Phi_4(z) = z + \bar{z} + c_0$ uses one conjugation and two additions:

$$|e_4| \le \epsilon_m + \epsilon_m + \epsilon_m = 3\epsilon_m.$$

□

D.2 Domains and Lipschitz Bounds

Proof of Lemma 2. Since $x \in \mathbb{R}$, every entry of $\widetilde{x}_1 + e_1 = \Phi_1(x)$ has magnitude one. The magnitude bounds in the proof of Proposition 4 give

$$\begin{aligned} 1+\delta &= \max\{1, \|\widetilde{x}_1\|_\infty, \|\widetilde{x}_1 + e_1\|_\infty, \|\widetilde{x}_2\|_\infty, \|\widetilde{x}_2 + e_2\|_\infty\} \\ &\le (1 + \|e_1\|_\infty)^t (1+\epsilon_m)^{4t} \le e^{t\|e_1\|_\infty + 4t\epsilon_m}. \end{aligned}$$

Since t is fixed and $\|e_1\|_\infty + \epsilon_m = o(1)$,

$$\delta = O(\|e_1\|_\infty + \epsilon_m) = O(\epsilon_{\mathsf{BTS}} + d^{1+1/t}\epsilon_m) = o(1), \qquad (1+\delta)^t = 1 + o(1).$$

□

Proof of Proposition 5. For $|a_j|, |b_j| \le 1+\delta$,

$$\begin{aligned} \prod_{j=1}^{t} a_j - \prod_{j=1}^{t} b_j &= \sum_{k=1}^{t} (a_k - b_k) \prod_{j<k} a_j \prod_{j>k} b_j, \\ \left| \prod_{j=1}^{t} a_j - \prod_{j=1}^{t} b_j \right| &\le t(1+\delta)^{t-1} \max_k |a_k - b_k|. \end{aligned}$$

Each nonconstant monomial of g_1 contains t factors, possibly conjugated. Using $|\bar{a} - \bar{b}| = |a - b|$ and then summing the coefficient magnitudes gives

$$\mathrm{Lip}_{\mathbb{D}_\delta^{nt}}(g_1) \le 2t(1+\delta)^{t-1}\|c\|_1 = (2t + o(1))\|c\|_1.$$

For g_2, let $(u, v), (u', v') \in \mathbb{D}_\delta^{n(R+S)}$. Since $\sum_{k,l} |C_{k,l}| = \|c\|_1$,

$$\begin{aligned}|u^\top C v - (u')^\top C v'| &\le |(u-u')^\top C v| + |(u')^\top C (v - v')| \\ &\le (1+\delta)\|c\|_1(\|u - u'\|_\infty + \|v - v'\|_\infty).\end{aligned}$$

The map $\Phi_4(z) = z + \bar{z} + c_0$ multiplies the preceding bound by at most two. Hence

$$\mathrm{Lip}_{\mathbb{D}_\delta^{n(R+S)}}(g_2) \le 4(1+\delta)\|c\|_1 = (4 + o(1))\|c\|_1.$$

For $t = 1$, $g_1 = g_2 : z \mapsto c^\top z + \overline{c^\top z} + c_0$, so both Lipschitz constants are at most $2\|c\|_1$. For the remaining two maps,

$$\begin{aligned}|g_3(z) - g_3(w)| &= |(z - w) + \overline{z - w}| \le 2|z - w|, \\ |g_4(z) - g_4(w)| &= |z - w|.\end{aligned}$$

Thus $\mathrm{Lip}(g_3) \le 2$ and $\mathrm{Lip}(g_4) = 1$. □

D.3 Numerical Error

Proof of Theorem 2. For $t \ge 2$, substitute the local error bounds and the Lipschitz bounds into (9):

$$\begin{aligned}|\widetilde{f}(x) - f(x)| \le\ & 2t(1+\delta)^{t-1}\|c\|_1\|e_1\|_\infty \\ & + 4(1+\delta)\|c\|_1\|e_2\|_\infty \\ & + \big[2n(1+\delta)\|c\|_1 + 8(1+\delta)d + 4nR + 4n - 1\big]\epsilon_m.\end{aligned}$$

Since $\delta = o(1)$, $n = \Theta(d^{1/t})$, $nR \le d$, and $\|c\|_1 = \Omega(1)$,

$$\begin{aligned}\eta_{\mathrm{num}}(x) &\le O(\|c\|_1)(\epsilon_{\mathsf{BTS}} + d^{1+1/t}\epsilon_m) + O\big((d^{1/t}\|c\|_1 + d)\epsilon_m\big) \\ &= O(\|c\|_1)\epsilon_{\mathsf{BTS}} + O(d^{1+1/t}\|c\|_1)\epsilon_m.\end{aligned}$$

For $t = 1$, $e_2 = 0$ and

$$\begin{aligned}|\widetilde{f}(x) - f(x)| &\le 2\|c\|_1\|e_1\|_\infty + (6d - 1)\epsilon_m \\ &= O(\|c\|_1)\epsilon_{\mathsf{BTS}} + O(d^2\|c\|_1)\epsilon_m.\end{aligned}$$

The bounds are uniform over $x \in \mathbb{R}$. In particular,

$$\epsilon_m = O(d^{-1-1/t}\epsilon_{\mathsf{BTS}}) \quad \Longrightarrow \quad \eta_{\mathrm{num}}(x) = O(\|c\|_1\epsilon_{\mathsf{BTS}}).$$

□

D.4 Input Error

Proof of Proposition 6. For $0 \le \ell < p$, define

$$\widehat{\varphi}_\ell := \frac{1}{p}\sum_{k=0}^{p-1} \varphi(k/p) e^{-2\pi \mathrm{i}\ell k/p}.$$

The Hermite coefficients are

$$c_0 = \widehat{\varphi}_0, \quad c_\ell = (1-\ell/p)\widehat{\varphi}_\ell \quad (1 \le \ell < p).$$

Since φ is real, $\overline{\widehat{\varphi}_{p-\ell}} = \widehat{\varphi}_\ell$ for $1 \le \ell < p$. The polynomial

$$f(u) = c_0 + \sum_{\ell=1}^{p-1} \left(c_\ell e^{2\pi \mathrm{i}\ell u} + \bar{c}_\ell e^{-2\pi \mathrm{i}\ell u} \right)$$

has degree at most $p-1$. For $1 \le \ell < p$,

$$c_\ell + \bar{c}_{p-\ell} = \widehat{\varphi}_\ell, \qquad \ell c_\ell - (p-\ell)\bar{c}_{p-\ell} = 0.$$

Grouping the frequencies modulo p therefore gives

$$f(k/p) = \sum_{\ell=0}^{p-1} \widehat{\varphi}_\ell e^{2\pi \mathrm{i}\ell k/p} = \varphi(k/p), \qquad f'(k/p) = 0.$$

Pairing ℓ and $p-\ell$ and applying Cauchy–Schwarz and Parseval's identity gives

$$\begin{aligned} \|c\|_1 &= \sum_{\ell=1}^{p-1} (1-\ell/p)|\widehat{\varphi}_\ell| = \frac{1}{2}\sum_{\ell=1}^{p-1} |\widehat{\varphi}_\ell| \\ &\le \frac{\sqrt{p-1}}{2} \left(\frac{1}{p}\sum_{k=0}^{p-1} \varphi(k/p)^2 - c_0^2 \right)^{1/2} \\ &\le \frac{\sqrt{p-1}}{2} \sqrt{c_0(1-c_0)} \le \frac{\sqrt{p-1}}{4}. \end{aligned}$$

The last two inequalities use $\varphi(k/p)^2 \le \varphi(k/p)$ and $0 \le c_0 \le 1$. For the second derivative, $|\widehat{\varphi}_\ell| \le 1$ gives

$$\sup_{u \in \mathbb{R}} |f''(u)| \le 8\pi^2 \sum_{\ell=1}^{p-1} \ell^2 (1-\ell/p) = \frac{2\pi^2}{3} p(p^2-1).$$

Fix $x \in I$ and choose real representatives with $|\widetilde{x} - x| < e_{\mathrm{pert}} < 1/(2p)$. Since $f(x) = \varphi(x)$ and $f'(x) = 0$, Taylor's theorem gives

$$\begin{aligned}
|f(\widetilde{x}) - \varphi(x)| &\le \frac{|\widetilde{x} - x|^2}{2} \sup_{|u-x| \le e_{\mathrm{pert}}} |f''(u)|, \\
|f(\widetilde{x}) - \varphi(x)| &\le \frac{e_{\mathrm{pert}}^2}{2} \sup_{\substack{x \in I \\ |u-x| \le e_{\mathrm{pert}}}} |f''(u)| \\
&\le \frac{e_{\mathrm{pert}}^2}{2} \sup_{u \in \mathbb{R}} |f''(u)| \le \frac{\pi^2}{3} p(p^2-1) e_{\mathrm{pert}}^2 = O(p^3 e_{\mathrm{pert}}^2).
\end{aligned}$$

The bound holds uniformly over $x \in I$. □

E Convex Optimization on the Target Interval

We describe the numerical procedure used to solve $C_d(\sigma)$ in Section 5. Write

$$f(x) = c_0 + \sum_{\ell=1}^{d} \big(a_\ell \cos(2\pi\ell x) + b_\ell \sin(2\pi\ell x)\big), \qquad \|c\|_1 = \frac{1}{2} \sum_{\ell=1}^{d} \sqrt{a_\ell^2 + b_\ell^2}.$$

Let $r_f(x) = f(x) - \varphi_\sigma(x)$ for absolute error and $r_f(x) = (f(x) - \varphi_\sigma(x))/\varphi_\sigma(x)$ for relative error. For a finite set $X \subset I_\sigma$, we solve the SOCP

$$\begin{aligned}
\underset{c_0, a_\ell, b_\ell, s_\ell}{\text{minimize}} \quad & \sum_{\ell=1}^{d} s_\ell \\
\text{subject to} \quad & |r_f(x)| \le \eta_{\mathrm{app}} - \delta \quad (x \in X), \\
& \sqrt{a_\ell^2 + b_\ell^2} \le 2 s_\ell \qquad (1 \le \ell \le d),
\end{aligned}$$

where $\delta > 0$ is a small numerical safety margin. The second-order cone constraints make the objective equal to $\|c\|_1$.

We solved the finite problems with the Clarabel conic optimization solver [20]. Local maxima were first identified on an oversampled grid and then refined by one-dimensional local maximization. The grid density was doubled until the estimated supremum stabilized. Each accepted polynomial was checked again using a dense grid and numerically located roots of r_f'. In our experiments we used $\delta = 10^{-8}$. We ran Algorithm 7 for every $(d, \sigma) \in \mathcal{D} \times \Sigma$ and, for each d, retained the feasible value of σ with the smallest coefficient norm. These checks are numerical and do not constitute an interval-arithmetic certificate for the continuous supremum.

Algorithm 7 Constraint generation for $C_d(\sigma)$

Require: Degree d, interval $I_\sigma = [-\sigma/2, \sigma/2]$, target φ_σ, and error bound η_{app}
$X \leftarrow \{(\sigma/2)\cos(j\pi/(16d)) : 0 \le j \le 16d\}$
repeat
  Solve (E) on X to obtain f
  Find the endpoints and local maximizers $\mathcal{E}$ of $|r_f|$ on I_σ
  $\widehat{U} \leftarrow \max_{x\in\mathcal{E}} |r_f(x)|$
  if $\widehat{U} \le \eta_{\text{app}}$ **then**
    Independently check the endpoints and stationary points of r_f
    return f and $\|c\|_1$
  end if
  $X \leftarrow X \cup \{x \in \mathcal{E} : |r_f(x)| > \eta_{\text{app}} - \delta\}$
until no new constraint is found or the iteration limit is reached
return undetermined